

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG



Laporan Tugas Besar

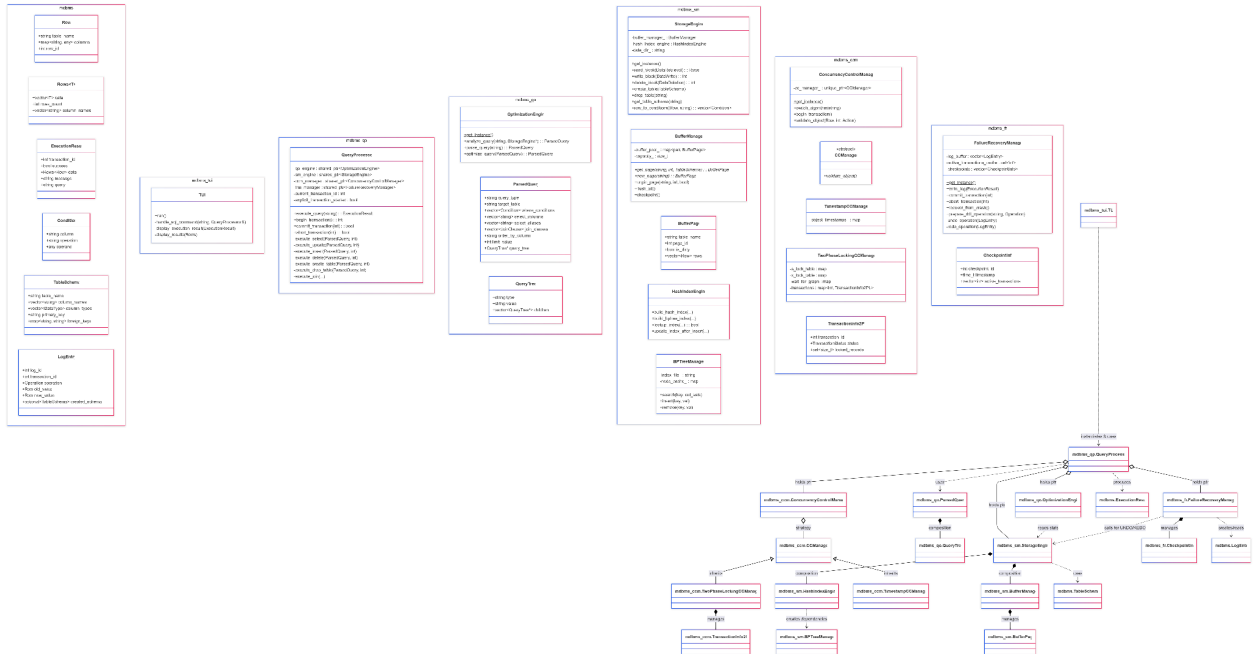
Mini Database Management System Development

Oleh:
SUPER GROUP 1
MahSQL

IF3140 - Sistem Basis Data
2025

Part I. Gambaran Umum DBMS

Database Management System (DBMS) atau Sistem Manajemen Basis Data adalah sebuah sistem perangkat lunak yang dirancang secara khusus untuk mengelola, menyimpan, dan mengambil data dari sebuah basis data secara efisien dan aman.



[Gambar selengkapnya \(Google Drive\)](#)

Berdasarkan class diagram di atas, rancangan mDBMS ini menggunakan arsitektur modular berbasis komponen yang sudah terstruktur. Sistem ini terdiri dari lima komponen inti yang dikoordinasi seluruhnya oleh QueryProcessor (QP). Seperti terlihat pada diagram relasi, QueryProcessor memegang referensi (*aggregation*) ke empat komponen lainnya:

1. OptimizationEngine (sebagai qo_engine)
2. StorageEngine (sebagai sm_engine)
3. ConcurrencyControlManager (sebagai ccm_manager)
4. FailureRecoveryManager (sebagai frm_manager)

Part II. Detail Komponen DBMS

Berikut adalah komponen-komponen yang diimplementasikan.

1. Query Processor

Komponen Query Processor (QP) bertindak sebagai "otak" atau koordinator pusat dari mDBMS. Komponen ini bertanggung jawab untuk menangani seluruh siklus hidup eksekusi query. Pada tugas besar ini component Query Processor diimplementasikan dalam Class QueryProcessor. Berikut daftar atribut dan method yang terdapat pada rancangan Class QueryProcessor yang telah dibuat.

Attribute

Nama	Tipe	Penjelasan
current_transaction_id	int	Menyimpan ID transaksi yang sedang aktif dan digunakan untuk memastikan tiap query berjalan dalam konteks transaksi yang benar.
explicit_transaction_started	bool	Menandakan apakah transaksi dimulai secara eksplisit dengan BEGIN (true) atau otomatis oleh QP (false).

Method

Fitur	Kode	Penjelasan
Eksekusi Query	<pre>ExecutionResult execute_query(const std::string& query);</pre>	Method utama Query Processor yang menangani parsing, optimasi, izin concurrency control, eksekusi query (SELECT/INSERT/UPDATE/DELETE), logging, dan manajemen transaksi.
BEGIN TRANSACTION	<pre>int begin_transaction();</pre>	Membuat transaksi baru melalui CCM dan mencatat log BEGIN melalui FRM.

COMMIT	<code>bool commit_transaction(int transaction_id);</code>	Menyelesaikan transaksi, menutup akses CCM, dan menerapkan perubahan permanen melalui FRM.
ABORT	<code>bool abort_transaction(int transaction_id);</code>	Melakukan rollback penuh melalui FRM dan mengakhiri transaksi di CCM.
SELECT	<code>Rows<Row> execute_select(const ParsedQuery&, int);</code>	Mengambil data berdasarkan FROM, JOIN, WHERE, ORDER BY, LIMIT, serta mendukung alias (AS).
UPDATE	<code>int execute_update(const ParsedQuery&, int);</code>	Mengubah baris dengan kondisi WHERE dan meminta izin WRITE dari CCM.
INSERT	<code>int execute_insert(const ParsedQuery&, int);</code>	Menyisipkan satu row baru, memetakan kolom ke tipe data schema, dan meminta akses WRITE.
DELETE	<code>int execute_delete(const ParsedQuery&, int);</code>	Menghapus baris berdasarkan kondisi dan mencatat perubahan untuk recovery.
CREATE TABLE	<code>bool execute_create_table(const ParsedQuery&, int);</code>	Membuat tabel baru dan metadata .schema.
DROP TABLE	<code>bool execute_drop_table(const ParsedQuery&, int);</code>	Menghapus file .schema, .dat, dan index terkait.
JOIN	<code>Rows<Row> execute_join(const Rows<Row>&, const Rows<Row>&, const Condition&, const std::string&);</code>	Melakukan INNER / NATURAL JOIN berdasarkan kondisi parsed query.
WHERE	<code>Rows<Row> apply_where_clause(const Rows<Row>&, const</code>	Memfilter baris dengan perbandingan literal atau kolom-ke-kolom.

	<code>vector<Condition>&);</code>	
ORDER BY	<code>Rows<Row> apply_order_by(const Rows<Row>&, const string&, bool);</code>	Mengurutkan hasil berdasarkan kolom tertentu.
LIMIT	<code>Rows<Row> apply_limit(const Rows<Row>&, int);</code>	Mengambil sejumlah baris pertama.
AS	<code>Rows<Row> apply_table_aliases(const Rows<Row>& rows, const std::string& table_name, const std::map<std::string, std::string>& table_aliases); std::string resolve_aliased_column(const std::string& column, const std::map<std::string, std::string>& table_aliases); std::string get_table_from_alias(const std::string& alias, const std::map<std::string, std::string>& table_aliases);</code>	Mengelola tabel dan kolom yang menggunakan alias.
Deteksi tipe query	<code>std::string parse_query_type(const std::string& query);</code>	Mengidentifikasi apakah query adalah SELECT/INSERT/UPDATE/DELETE/BEGIN/COMMIT/ABORT.

2. Query Optimizer

Query Optimizer berperan untuk menerima sebuah *query* sebagai input dari *Query Processor*, kemudian melakukan *parsing* dan validasi, serta mengembalikan *query plan* yang telah dioptimasi dalam bentuk *parsed query*

object. Terdapat tiga fungsi utama yang menjadi dasar dari *Query Optimizer*, yakni `parse_query(query:str) → ParsedQuery` yang menerima *query* dalam bentuk *string* dan mengubahnya menjadi *object* yang merepresentasikan *query* yang telah di-*parse*; `optimize_query(query:ParsedQuery) → ParsedQuery` yang melakukan optimasi pada *parsed query* berdasarkan aturan optimisasi, kemudian mengembalikan *query* yang telah di-*optimize*; serta `get_cost(query:ParsedQuery) → int` yang menghitung biaya eksekusi dari *query* yang diberikan dan sebagai metode pendukung untuk fungsi `optimize_query`. Berikut adalah bagian yang sudah diimplementasikan.

Fitur	Kelas/Fungsi/Prosedur	Penjelasan
Deteksi tipe <i>query</i>	<pre>std::string detect_query_type(const std::string& normalized_query) ParsedQuery sql_parser(const std::string& query)</pre>	Mendeteksi <i>prefix</i> (SELECT/INSERT/UPDATE), lalu memanggil <i>parser</i> sesuai tipenya.
<i>Parsing</i> SELECT (select <i>list</i> , FROM/JOIN, WHERE, ORDER BY, LIMIT)	<pre>void parse_select_query(const std::string& query, ParsedQuery& pq) void parse_from_clause(const std::string& clause, ParsedQuery& pq)</pre>	Mengambil select <i>list</i> FROM <i>multi-table</i> dengan ON sederhana, rekam <i>table</i> alias dan join_pairs, WHERE, ORDER BY, dan LIMIT.
<i>Parsing</i> UPDATE (SET, WHERE)	<pre>void parse_update_query(const std::string& query, ParsedQuery& pq)</pre>	Mengambil target tabel, pecah SET kolom=literal dan WHERE, kemudian mengisi <i>target_table</i> , <i>set_values</i> , dan <i>where_conditions</i> .
<i>Parsing</i> INSERT (columns, VALUES)	<pre>void parse_insert_query(const std::string& query,</pre>	Mengambil target label, VALUES, kemudian mengisi <i>target_table</i> ,

	ParsedQuery& pq)	insert_columns, insert_values.
<i>Parsing</i> DELETE (WHERE)	void parse_delete_query(const std::string& query, ParsedQuery& pq)	Mengambil target tabel dan WHERE, kemudian mengisi target_table, where_conditions.
<i>Parsing</i> CREATE TABLE (kolom, PK/FK, length)	void parse_create_table(const std::string& query, ParsedQuery& pq)	Mengambil nama tabel dan isinya.
<i>Parsing</i> DROP TABLE	void parse_drop_table(const std::string& query, ParsedQuery& pq)	Mengambil nama tabel setelah DROP TABLE dan mengisi target_table.
API untuk <i>Query Optimizer</i>	ParsedQuery OptimizationEngine::pars e_query(const std::string& query) ParsedQuery OptimizationEngine::opti mize_query(const ParsedQuery& query, const mdbms::sm::StorageEngine * storage) ParsedQuery OptimizationEngine::anal yze_query(const std::string& query, const mdbms::sm::StorageEngine * storage) ParsedQuery sql_parser(const std::string& query)	parse_query memanggil sql_parser kemudian membangun QueryTree melalui plan_tree. get_cost menghitung biaya dengan memanggil estimate_cost pada query_tree
Struktur data (<i>plan</i> dan <i>query</i>) untuk	struct QueryTree	QueryTree menyimpan jenis operasi, nilai,

<i>parsing dan optimization</i>	<code>struct ParsedQuery</code>	<i>children</i> , dan <i>parent</i> . ParsedQuery menyimpan komponen SELECT/FROM/WHERE/JOIN serta <code>query_tree</code> dan <code>estimated_cost</code>
Membangun plan SELECT (SCAN, JOIN, SELECT/WHERE, PROJECT) dan menangani non-SELECT (CREATE/INSERT/UPDATE/DELETE/DROP/BEGIN)	QueryTree* <code>plan_tree(const ParsedQuery& parsed)</code>	Membangun rantai SCAN per tabel, JOIN antar tabel, lapisan SELECT untuk setiap kondisi WHERE, lalu PROJECT berisi <i>select-list</i> sebagai akarnya. Jika <code>query_type</code> bukan SELECT, buat satu QueryTree root dengan type diisi <code>query_type</code> .
Memecah kondisi AND menjadi seleksi berantai	QueryTree* <code>splitting_conjunction(QueryTree* plan, const std::vector<Condition>& where_conditions)</code>	Pada <i>node</i> SELECT yang berisi ekspresi dengan AND, dilakukan <i>split</i> menjadi dua <i>node</i> SELECT bertingkat, lalu direkursi untuk membuat setiap kondisi berdiri sendiri untuk <i>pushdown</i> .
Mengurutkan seleksi berdasarkan perkiraan selektivitas	QueryTree* <code>reorder_selections(QueryTree* plan, const std::vector<Condition>& where_conditions)</code>	Mengumpulkan rantai SELECT, mencocokkan <i>string</i> nilai <i>node</i> SELECT dengan kondisi di <code>where_conditions</code> , kemudian melakukan <code>estimate_selectivity</code> . SELECT dengan selektivitas tinggi dieksekusi lebih awal.
<i>Pushdown selection</i> ke sisi JOIN yang relevan	QueryTree* <code>pushdown_selection(QueryTree* plan, const std::vector<Condition>& where_conditions, const std::vector<std::string>& from_tables)</code>	Jika SELECT berada di atas JOIN dan kondisinya hanya menyentuh tabel di sisi kiri atau kanan, SELECT dipindah ke cabang tersebut untuk mengurangi data lebih awal.

Menghapus/menggabungkan proyeksi redundan	<pre>QueryTree* redundant_projection(QueryTree* plan, const std::vector<std::string> & select_columns)</pre>	Mendeteksi PROJECT berlapis dengan set kolom sama/subset. Jika identik, <i>node</i> PROJECT atas/bawah akan digabung atau dihapus. Dilakukan juga penghapusan PROJECT yang hanya memproyeksi persis kolom yang memang dipakai <i>subtree</i> di bawahnya.
Pengaplikasian urutan aturan	<pre>QueryTree* apply_optimizer_rules(QueryTree* plan, const std::vector<Condition>& where_conditions, const std::vector<std::string> & from_tables, const std::vector<std::string> & select_columns)</pre>	Memanggil <i>splitting_conjunction</i> → <i>reorder_selections</i> → <i>pushdown_selection</i> → <i>redundant_projection</i> , lalu <i>passthrough</i> .
Estimasi biaya per <i>node plan</i> (SCAN/SELECT/PROJECT/JOIN)	<pre>int estimate_cost(const QueryTree& root, const mdbms::sm::StorageEngine * storage)</pre>	Melakukan <i>traversal tree</i> , membangun <i>map</i> alias → tabel, lalu menghitung <i>cost/rows/blocks</i> untuk setiap <i>node</i> , hasil akhir yang dikembalikan berupa integer.
Melakukan SCAN dengan statistik <i>storage</i>	<pre>CostEstimate estimate_scan(const QueryTree* node, const std::unordered_map<std::string, std::string>& alias_map, const mdbms::sm::StorageEngine * storage, const Predicate* predicate)</pre>	Mengambil statistik tabel melalui <i>lookup_stats</i> . Menghitung <i>rows base</i> , menerapkan selektivitas <i>predicate</i> jika ada. Biaya hasil adalah <i>block sequential</i> ditambah CPU per <i>tuple</i> . Hasil akhir yang disimpan adalah <i>rows</i> , <i>cost</i> , <i>blocks</i> , dan jumlah <i>rows</i> per tabel.
Estimasi seleksi (WHERE)	<pre>CostEstimate estimate_select(const QueryTree* node, const std::unordered_map<std::string,</pre>	Melakukan <i>parse</i> ekspresi SELECT menjadi Predicate. Selektivitas dihitungkan melalui

	<pre>string, std::string>& alias_map, const mdbms::sm::StorageEngine * storage)</pre>	predicate_selectivity.
Estimasi proyeksi	<pre>CostEstimate estimate_project(const QueryTree* node, const std::unordered_map<std:: string, std::string>& alias_map, const mdbms::sm::StorageEngine * storage)</pre>	Mengestimasi nilai <i>child</i>, kemudian menambahkan nilai CPU per <i>tuple</i> untuk <i>projection</i>.
Estimasi JOIN	<pre>CostEstimate estimate_join_node(const QueryTree* node, const std::unordered_map<std:: string, std::string>& alias_map, const mdbms::sm::StorageEngine * storage)</pre>	Melakukan <i>parsing</i> label JOIN menjadi JoinCondition, lalu mengestimasi <i>child</i> kiri/kanan dan menghitung <i>selectivity join</i> untuk <i>equality</i>. Dilakukan perhitungan biaya untuk beberapa strategi, yakni <i>nested loop</i>, <i>hash join</i>, <i>merge</i>, kemudian mengambil yang minimum.

Komponen *Query Optimizer* telah menyelesaikan empat bagian utama, yakni *parsing*, pembangunan *query tree*, *rule-based optimization*, dan estimasi biaya. Modul *parsing* mampu memproses *query* SELECT secara lengkap, termasuk klausa SELECT, FROM, JOIN, dan WHERE. Hasil *parsing* dirangkum dalam objek `ParsedQuery` yang digunakan sebagai *input* untuk tahap berikutnya. Selanjutnya, *Query Optimizer* juga telah membangun `QueryTree` yang secara sistematis menggambarkan bagaimana sebuah *query* dieksekusi, sehingga dapat dioptimasi lebih lanjut.

Proses optimasi juga telah berfungsi melalui serangkaian *optimizer rules* seperti pemecahan kondisi konjungtif, *reordering selection*, dan *pushdown selection*. Aturan-aturan ini menghasilkan versi *query plan* yang lebih efisien.

Modul *cost estimation* juga telah diselesaikan dan terintegrasi dengan *Storage Manager*. Dengan demikian, sistem kini dapat menentukan rencana eksekusi dengan biaya terendah.

3. Concurrency Control Manager

Komponen Concurrency Control Manager (CCM) bertindak sebagai regulator bagi eksekusi transaksi data dalam mDBMS. Komponen ini bertanggung jawab penuh dalam mengatur *scheduling query* dan menangani konkurensi. Tujuan utamanya adalah untuk memastikan properti ACID (khususnya *isolation*) terpenuhi dalam setiap transaksi dan untuk mencegah terjadinya *deadlock*.

Sesuai dengan spesifikasi tugas besar, desain utama komponen ini diimplementasikan dalam sebuah singleton class *ConcurrencyControlManager* yang memastikan hanya ada satu *instance* CCM yang aktif dalam mengelola seluruh transaksi dari banyak *client* sekaligus. Selain itu, alur algoritma dari setiap protokol didasarkan terhadap status yang mungkin dimiliki oleh transaksi, meliputi status *ACTIVE*, *PARTIALLY COMMITTED*, *COMMITTED*, *FAILED*, *ABORTED*, dan *TERMINATED*. Algoritma dari setiap protokol pada Concurrency Control Manager bertanggung jawab dalam memvalidasi dan melakukan perubahan status untuk menentukan alur keputusan (*commit*, *abort*, atau *wait*) dalam setiap skenario eksekusi transaksi.

Berikut adalah bagian-bagian yang diimplementasikan oleh komponen Concurrency Control Manager.

Fitur	Kode	Penjelasan
Milestone 1		
<i>Interface class</i> ConcurrencyControlManager	<pre> class ConcurrencyControlManager { public: int begin_transaction(); void log_object(const Row& object, int transaction_id); Response validate_object(const Row& object, int transaction_id, Action action); </pre>	<i>Interface class</i> ConcurrencyControlManager dibentuk mengikuti panduan dalam spesifikasi, dengan tambahan fungsi <i>set_protocol</i> yang mengubah protokol yang digunakan oleh CCM.

	<pre> void end_transaction(int transaction_id); void set_protocol(const std::string& protocol); }; </pre>	
Struktur tipe data Row	<pre> struct Row { std::string table_name; std::map<std::string, std::any> columns; int row_id; Row() : row_id(-1) {} }; </pre>	Tipe data Row dibentuk sesuai dengan kebutuhan dalam implementasi CCM selanjutnya. Tipe data ini akan disesuaikan lebih lanjut, khususnya menyesuaikan dengan implementasi dari QP.
Struktur tipe data Response	<pre> struct Response { bool allowed; int transaction_id; std::string reason; Response() : allowed(false), transaction_id(-1) {} Response(bool allow, int tid) : allowed(allow), transaction_id(tid) {} }; </pre>	Tipe data Response menyatakan respons dari CCM terhadap sebuah transaksi, apakah diizinkan atau tidak. Response yang menolak transaksi akan memiliki alasan penolakan pada atribut reason.
Enumerasi tipe data Action	<pre> enum class Action { READ, WRITE }; </pre>	Enumerasi tipe data Action menyatakan operasi yang dilakukan oleh sebuah transaksi, dapat berupa membaca (<i>read</i>) atau menulis (<i>write</i>).
Enumerasi tipe data TransactionStatus	<pre> enum class TransactionStatus { ACTIVE, PARTIALLY_COMMITTED, COMMITTED, FAILED, ABORTED, TERMINATED }; </pre>	Enumerasi tipe data TransactionStatus menyatakan status dari sebuah transaksi mengikuti panduan dari spesifikasi.
Milestone 2		
Perbaikan <i>class</i> ConcurrencyControlManager	<pre> class ConcurrencyControlManager { public: ConcurrencyControlManager(const ConcurrencyControlManager&) = delete; ConcurrencyControlManager& operator=(const </pre>	Implementasi <i>class</i> ConcurrencyControlManager disesuaikan untuk menerapkan <i>design pattern Singleton</i> . ConcurrencyControlMan

	<pre> ConcurrencyControlManager&) = delete; ~ConcurrencyControlManager(); // Method untuk mendapatkan instance singleton static ConcurrencyControlManager& get_instance(const std::string& algorithm = "timestamp"); void switch_algorithm(const std::string& algorithm); int begin_transaction(); void log_object(const Row& object, int transaction_id); Response validate_object(const Row& object, int transaction_id, Action action); void end_transaction(int transaction_id); private: // Constructor private untuk Singleton explicit ConcurrencyControlManager(const std::string& algorithm = "timestamp"); std::unique_ptr<CCManager> cc_manager_; std::string current_algorithm_; static std::unique_ptr<ConcurrencyControlMa nager> instance_; static std::mutex instance_mutex_; }; </pre>	ager memiliki atribut cc_manager sebagai <i>instance</i> global yang mengatur protokol <i>concurrency control</i> yang digunakan
<i>Class</i> CCManager	<pre> class CCManager { public: virtual ~CCManager() = default; virtual int begin_transaction() = 0; virtual void log_object(const Row& object, int transaction_id) = 0; virtual Response validate_object(const Row& object, </pre>	<i>Class</i> CCManager berperan sebagai <i>abstract base class</i> (ABC) dari implementasi spesifik protokol <i>concurrency control</i>

	<pre> int transaction_id, Action action) = 0; virtual void end_transaction(int transaction_id) = 0; protected: static size_t generate_row_hash(const Row& row); }; </pre>	
<i>Class</i> TimestampCCManager	<pre> class TimestampCCManager : public CCManager { public: TimestampCCManager(); ~TimestampCCManager() override; int begin_transaction() override; void log_object(const Row& object, int transaction_id) override; Response validate_object(const Row& object, int transaction_id, Action action) override; void end_transaction(int transaction_id) override; private: std::map<int, std::shared_ptr<Transaction>> transactions_; std::map<size_t, ObjectTimestamp> object_timestamps_; int current_timestamp_; std::mutex mutex_; }; </pre>	<i>Class</i> TimestampCCManager merupakan implementasi <i>concrete class</i> yang mewarisi <i>class</i> CManager untuk <i>timestamp protocol</i>
<i>Class</i> TwoPhaseLockingCCManager	<pre> class TwoPhaseLockingCCManager : public CManager { public: TwoPhaseLockingCCManager(); ~TwoPhaseLockingCCManager() override; int begin_transaction() override; void log_object(const Row& object, int transaction_id) override; Response validate_object(const Row& object, int transaction_id, Action action) override; </pre>	<i>Class</i> TwoPhaseLocking merupakan implementasi <i>concrete class</i> yang mewarisi <i>class</i> CManager untuk <i>timestamp protocol</i>

	<pre> void end_transaction(int transaction_id) override; private: bool acquire_s_lock(int transaction_id, size_t record_hash); bool acquire_x_lock(int transaction_id, size_t record_hash); bool check_cycle(int current_id, std::set<int>& visited, std::set<int>& rec_stack); bool detect_deadlock(int waiting_trx_id, int holding_trx_id); void abort_transaction(int transaction_id); void release_s_lock(int transaction_id, size_t record_hash); void release_x_lock(int transaction_id, size_t record_hash); void release_all_locks(int transaction_id); std::map<size_t, int> x_lock_table; std::map<size_t, std::set<int>> s_lock_table; std::map<int, std::shared_ptr<TransactionInfo2PL>> transactions; std::map<int, std::set<int>> wait_for_graph; int current_transaction_id; std::recursive_mutex mutex_; }; </pre>	
<i>Unit testing</i> untuk implementasi <i>timestamp</i> <i>ordering</i>	<pre> void test_timestamp_singleton() void test_timestamp_thread_safe_singleton () void test_timestamp_baic_transaction_lifec ycle() void test_timestamp_read_after_write_conf lict() void test_timestamp_write_after_read_conf lict() void test_timestamp_concurrent_transactio ns() </pre>	Pengujian dari implementasi <i>timestamp</i> <i>ordering</i>

	<pre>void test_timestamp_invalid_txn_id() void test_timestamp_switch_algorithm()</pre>	
Unit testing untuk implementasi <i>two phase locking</i>	<pre>void test_twoPL_cc() void test_twoPL_abort() void test_twoPL_deadlock() void test_twoPL_deadlock_abort()</pre>	Pengujian dari implementasi <i>two phase locking</i>
Struktur tipe data Transaction	<pre>struct Transaction { int id; int timestamp; TransactionStatus state; Transaction(int tid, int ts) : id(tid), timestamp(ts), state(TransactionStatus::ACTIVE) {} };</pre>	Representasi transaksi beserta <i>timestamp</i> untuk <i>timestamp protocol</i>
Struktur tipe data ObjectTimestamp	<pre>struct ObjectTimestamp { int read_ts; int write_ts; ObjectTimestamp() : read_ts(0), write_ts(0) {} };</pre>	Representasi <i>read-timestamp</i> dan <i>write-timestamp</i> untuk sebuah objek / <i>record</i>
Enumerasi tipe data TransactionPhase	<pre>enum class TransactionPhase { GROWING, SHRINKING };</pre>	Representasi fase transaksi dalam <i>two phase locking protocol</i> , yaitu <i>growing</i> dan <i>shrinking</i>
Struktur tipe data TransactionInfo2PL	<pre>struct TransactionInfo2PL { int id; TransactionStatus status; TransactionPhase phase; std::set<size_t> locked_records; TransactionInfo2PL(int tid) : id(tid), status(TransactionStatus::ACTIVE), phase(TransactionPhase::GROWING) {} };</pre>	Representasi transaksi beserta fase transaksi dan set <i>record</i> yang dikunci oleh transaksi untuk <i>two phase locking protocol</i>
Milestone 3		
Fungsi <code>get_transaction_status</code> untuk	<pre>TransactionStatus get_transaction_status(int transaction_id);</pre>	Fungsi <code>get_transaction_status</code> mengembalikan <i>status</i>

setiap implementasi protokol		dari sebuah transaksi berdasarkan nilai <code>transaction_id</code>
Penambahan atribut <code>lock_wait_queue</code> untuk <i>class TwoPhaseLockingCCManager</i>	<code>std::map<size_t, std::list<int>>></code> <code>lock_wait_queue;</code>	Atribut <code>lock_wait_queue</code> menyimpan antrian ID transaksi yang menunggu akses terhadap sebuah <i>hash number</i> representasi sebuah <i>record</i>
Fungsi <code>execute_with_retry</code> sebagai fungsi <i>helper</i> untuk <i>unit testing</i> pemrosesan banyak <i>query</i> secara <i>multithreading</i>	<code>void execute_with_retry(ConcurrencyControlManager& ccm, int trx_id, const Row& row, Action action, std::string label)</code>	Fungsi <code>execute_with_retry</code> akan terus mencoba menjalankan operasi yang awalnya ditolak (untuk kondisi selain <i>aborted</i>). Fungsi ini menggambarkan pemanggilan fungsi yang akan dilakukan oleh <i>group query processor</i> nantinya
<i>Unit testing</i> untuk pengujian <i>concurrency control</i> untuk banyak <i>query</i> sekaligus secara <i>multithreading</i>	<code>void test_twoPL_multithreaded()</code> <code>void test_twoPL_queue_fairness()</code>	Fungsi <code>multithreaded</code> melakukan pengujian pemanggilan <i>multithreading</i> , sedangkan fungsi <i>queue_fairness</i> melakukan pengujian implementasi dari <i>waiting queue</i> untuk <i>two phase locking protocol</i>

Pada *milestone 1*, alur kerja integrasi difokuskan untuk mencapai kesepakatan desain dengan komponen lain, terutama penyelarasan struktur data *Row* yang akan digunakan dalam komunikasi dengan *Query Processor*.

Pada *milestone 2*, *group* telah merealisasikan implementasi dari protokol berbasis *timestamp* dan protokol berbasis *lock* (2PL). Implementasi dari kedua protokol memerlukan koordinasi lebih lanjut dengan *Query Processor* dan *Failure Recovery* mengenai langkah yang diambil dalam penanganan kondisi *deadlock*,

rollback transaksi, dan berbagai mekanisme-mekanisme lainnya yang dibutuhkan dalam menjalankan algoritma konkurensi.

Pada *milestone* 3, *group* telah memperbaiki implementasi kedua protokol yang telah diimplementasikan (protokol berbasis *timestamp* dan protokol berbasis *lock* (2PL)) untuk mengatasi kekurangan dalam *milestone* sebelumnya. Kedua protokol kini mendukung eksekusi transaksional untuk pemrosesan banyak *query* sebagai satu kesatuan dan sudah layak untuk diintegrasikan sepenuhnya dengan komponen mDBMS lainnya.

Integrasi mDBMS secara menyeluruh dilakukan melalui komponen *query processor* yang memanggil fungsi `get_instance()` dari *class* `ConcurrencyControlManager` untuk mengembalikan *instance* Singleton untuk kemudian memanggil fungsi yang dibutuhkan. Contoh pemanggilan *instance* adalah sebagai berikut.

```
ExecutionResult QueryProcessor::execute_query(const std::string& query) {  
    ...  
  
    // Auto-commit queries only if transaction was auto-started (not explicitly  
    started with BEGIN)  
    if (!explicit_transaction_started) {  
  
        mdbms::ccm::ConcurrencyControlManager::get_instance().end_transaction(result.transaction_id);  
        current_transaction_id = -1;  
    }  
  
    ...  
}
```

4. Storage Manager

Komponen Storage Manager bertanggung jawab pada abstraksi seluruh operasi I/O disk. Seluruh fungsionalitas diimplementasikan pada suatu kelas `StorageEngine`, yang akan menyediakan interface untuk digunakan `Query Processor`. Interface ini terdiri dari beberapa metode diantaranya `read_block()`, `write_block()`, dan `delete_block()`. Pada implementasi ini, data akan disimpan

dalam file biner (.dat) yang terpisah untuk setiap tabelnya. Implementasi ini sangat bergantung pada helper method untuk metadata (getSchema) dan juga serializer.

Pada milestone 1, telah diimplementasikan fungsionalitas read_block(), write_block(), dan delete_block() pada level biner. Inti dari implementasi ini adalah pada helper method serialize_row() dan deserialize_row(). Metode serialize_row() akan mentranslasikan objek Row dari representasi logis di memori ke format biner row-based di disk, sebaliknya deserialize_row() akan mentranslasikan objek dari disk agar bisa di proses di memori. Format ini menangani tipe fixed-length (INTEGER, FLOAT) secara langsung dan tipe variable-length (VARCHAR, CHAR) menggunakan Length-Prefix. Metode write_block() mengimplementasikan logika INSERT (melalui append) dan UPDATE (melalui read-modify-write). Baik UPDATE maupun delete_block() menggunakan file temporer untuk menjamin atomisitas dan mencegah terjadinya data corrupt. Selain itu, telah diimplementasikan fungsi check_conditions(), yang mengevaluasi semua kondisi pada vector<Condition>.

Pada milestone 2, diimplementasikan fungsi set index dengan tipe index hash yang akan disimpan pada file dengan format <nama-tabel>.<nama-kolom>.hashidx. Penyesuaian terhadap fungsi read, write, dan delete juga dilakukan agar *compatible* dengan pencarian dan pembaruan index.

Pada milestone 3, diimplementasikan fungsi fungsi yang dibutuhkan untuk pembuatan dan penghapusan schema tabel baru. Schema tabel disimpan dalam file dengan format <nama-tabel>.schema dan terdapat file tables.meta yang berisi jumlah serta semua nama dari tabel yang ada saat ini.

Nama Fungsi	Masukan	Luaran	Deskripsi
-------------	---------	--------	-----------

read_block	DataRetrieval	Rows<Row>	Melakukan pembacaan data dari file penyimpanan menggunakan index scan jika memungkinkan, atau full scan jika tidak.
read_using_offsets	DataRetrieval	Rows<Row>	Membaca baris tertentu pada file data berdasarkan offset yang diperoleh dari index.
full_scan	DataRetrieval	Rows<Row>	Melakukan pembacaan seluruh baris dalam file tabel tanpa index (table scan).
write_block	DataWrite<Row>	int	Menulis data baru ke file (operasi INSERT) atau memodifikasi data yang ada (operasi UPDATE).
delete_block	DataDeletion	int	Menghapus data dari file biner yang sesuai dengan kriteria kondisi.
getSchema	std::string	TableSchema	Mengembalikan metadata skema (nama kolom, tipe data, dll.) untuk sebuah tabel.
serialize_row	std::ostream, Row, TableSchema	void	Menerjemahkan objek Row dari format memori (RAM) ke format biner untuk disimpan di file.
deserialize_row	std::istream, TableSchema	Row	Menerjemahkan data dari format biner di file kembali ke objek Row di memori (RAM).
check_conditions	Row, std::vector<Condition>	bool	Mengevaluasi apakah sebuah baris data memenuhi semua kondisi filter (klausa WHERE).
set_index	std::string& table, std::string& column, IndexType index_type	void	Melakukan indexing pada nama tabel dan kolom yang diinput dengan tipe index sesuai dengan masukan IndexType (dengan pilihan HASH atau BTREE).
build_hash_index	TableSchema& schema, std::string& table, std::string& column	void	Membangun index bertipe hash pada schema, tabel, dan kolom yang diinput pada parameter. Hasil indexing disimpan dalam file .hashindex

lookup_index	std::string& table, std::string& column, std::any& operand, std::vector<int64_ t>& out_offsets	bool	Mengecek apakah index tersedia dan menjalankan proses lookup yang sesuai.
lookup_hash	std::string& table, std::string& column, std::any& operand, std::vector<int64_ t>& out_offsets	bool	Mencari value dalam file index hash dan mengembalikan offset baris yang cocok.
update_index_a fter_insert	std::string& table, std::string& column, int64_t row_offset, Row& row	void	Memperbarui file index setelah operasi INSERT, yaitu dengan menambahkan entry key→offset untuk data baru.
update_index_a fter_update	std::string& table, std::string& column, int64_t row_offset, Row& old_row, Row& new_row	void	Digunakan ketika terjadi UPDATE pada baris yang memiliki index. Jika kolom index berubah, offset harus dipindah ke bucket baru. Jika tidak berubah, tidak melakukan apa-apa.
update_index_a fter_delete	std::string& table, std::string& column, int64_t row_offset, Row& old_row	void	Memperbarui file index setelah operasi DELETE dengan menghapus offset baris yang hilang dari bucket index.
any_to_int32	std::any &a, int32_t &out	bool	Melakukan konversi aman dari nilai std::any ke tipe int32_t. Fungsi ini hanya mendukung tipe integer yang umum: int32_t, int, int64_t, dan uint32_t. Jika tipe tidak cocok atau konversi gagal, fungsi mengembalikan false.
any_to_float	std::any &a, float &out	bool	Mengubah nilai std::any menjadi float. Fungsi ini mendukung dua tipe numerik: float dan double. Jika tipe tidak cocok atau

			konversi gagal, fungsi mengembalikan false.
any_to_string	std::any &a, std::string &out	bool	Mengonversi nilai std::any menjadi string. Fungsi ini mendukung dua tipe: std::string dan const char*.
get_tables	void	std::vector<TableSchema>	Mendapatkan semua table yang ada dengan membaca tables.meta yang berisi jumlah tabel yang ada beserta namanya
get_table_schema	std::string	TableSchema	Mendapatkan schema dari suatu table dengan membaca <nama_table>.schema yang berisi struktur schema dari suatu tabel
get_stats	void	std::map<std::string, Statistic>	Mendapatkan statistik dari semua table
create_table	const TableSchema& schema	bool	Menerima TableSchema dan membentuk file .schema yang berisi struktur dari schema yang diberikan
check_and_update_tables_metadata	const TableSchema& schema	bool	Helper function untuk create_table() yang berfungsi untuk mengupdate tables.meta terhadap tabel yang baru dibuat sekaligus untuk pengecekan apakah tabel sudah ada atau belum
delete_table	const TableSchema& schema	bool	Menerima TableSchema untuk menghapus file .schema, .dat, serta segala file indexing yang dibuat. Selain itu juga mengupdate tables.meta untuk penghapusan yang terjadi

Pada milestone 4, diimplementasikan BufferManager sebagai realisasi dari *data buffer* yang terdapat pada Storage Manager DBMS pada umumnya. Sebelumnya StorageEngine langsung mengakses *disk* dalam mengoperasikan blok data,

namun kini dengan adanya BufferManager sebagai perantara StorageEngine dan *disk*. Seluruh method yang relevan pada BufferManager pun akhirnya diintegrasikan dengan berbagai fungsi pada StorageEngine. Berikut adalah kumpulan fungsi baru yang terdapat pada BufferManager:

Nama Fungsi	Masukan	Luaran	Deskripsi
register_schema	table_name, schema	void	Cache schema table untuk efficient access saat deserialize
get_max_page_id	table_name	int	Cari max page_id dari buffer + disk untuk scan range
get_page	table_name, page_id, schema	BufferPage*	Load/retrieve page dari buffer atau disk, increment pin_count
new_page	table_name	BufferPage*	Buat page baru untuk INSERT, mark dirty = true
unpin_page	table_name, page_id, mark_dirty	void	Release reference, optionally mark dirty untuk UPDATE
flush_all		void	Flush semua dirty pages ke disk (saat shutdown)
flush_page	table_name, page_id	void	Flush single page jika dirty
evict_page		void	LRU eviction: cari unpinned page tertua, flush jika dirty, remove dari buffer
evict_table_without_flush	table_name	void	Evict semua pages table tanpa flush (untuk delete_table), discard dirty flag
checkpoint		void	Flush semua dirty pages (tidak evict), digunakan FRM
is_near_full		bool	Check buffer $\geq$ 80% kapasitas
clear_buffer		void	Clear buffer pool + hapus .dat files (untuk testing)

load_page_intern	table_name, page_id, page, schema	void	Load page dari disk, deserialize rows
flush_page_intern	page, schema	void	Serialize rows to buffer, write ke disk dengan pre-allocation
serialize_row_to_buffer	buffer, row, schema	void	Convert Row ke binary format (INTEGER/FLOAT/VARCHAR)
deserialize_row_to_buffer	ptr, end, schema	Row	Convert binary ke Row, check padding untuk detect EOF

Untuk integrasi, kelas StorageEngine akan berinteraksi langsung dengan kelas QueryProcessor. QueryProcessor akan memberikan objek yang diminta, selanjutnya StorageEngine akan mengeksekusi permintaan secara fisik terhadap file, dan mengembalikan hasilnya (hasil bisa Rows atau int row affected).

5. Failure Recovery

Komponen Failure Recovery Manager (FRM) bertugas untuk menangani pencatatan (logging) dan recovery terhadap eksekusi DBMS. Komponen ini bertanggung jawab penuh untuk mencatat setiap operasi database ke dalam write-ahead log, mengelola checkpoint secara periodik, dan melakukan recovery ketika terjadi transaction aborts atau kegagalan sistem. Tujuan utamanya adalah untuk memastikan durability dan atomicity dalam properti ACID, serta mengembalikan database ke kondisi konsisten setelah terjadi system crash.

Desain utama komponen ini diimplementasikan dalam sebuah class FailureRecoveryManager, yang mengelola log buffer, checkpoint history, dan proses recovery berdasarkan kriteria waktu atau transaction ID tertentu.

Kelas	Kode	Fungsi/Deskripsi
FailureRecoveryManager	<pre>class FailureRecoveryManager { public: static FailureRecoveryManager& get_instance(); FailureRecoveryManager(const FailureRecoveryManager&) =</pre>	Kelas ini berfungsi sebagai pusat kendali proses pencatatan log, pembuatan checkpoint, dan pemulihan. Kelas FailureRecoveryManager memiliki beberapa method yaitu:

	<pre> delete; FailureRecoveryManager& operator=(const FailureRecoveryManager&) = delete; ~FailureRecoveryManager(); void write_log(const ExecutionResult& info); void save_checkpoint(); void recover(const RecoverCriteria& criteria); void commit_transaction(int transaction_id); void abort_transaction(int transaction_id); void prepare_ddl_operation(const std::string& table_name, Operation op); private: FailureRecoveryManager(); const size_t MAX_BUFFER_SIZE = 50; std::vector<LogEntry> log_buffer; std::set<int> active_transactions_cache; std::vector<CheckpointInfo> checkpoints; std::string log_file_path; std::mutex mtx; int next_log_id; int next_checkpoint_id; sm::StorageEngine& storage_engine_; std::unordered_map<std::string, TableSchema> pending_drop_schemas_; std::thread checkpoint_thread_; std::atomic<bool> running_; const int </pre>	1. get_instance() : Mengembalikan single instance dari FailureRecoveryManager (Singleton pattern). 2. write_log(info: ExecutionResult) : Mencatat operasi database ke log buffer. Menentukan tipe operasi (INSERT/UPDATE/DELETE/BEGIN/COMMIT/ABORT), menyimpan old_value dan new_value, lalu menambahkan ke buffer. Memanggil flush jika buffer mencapai kapasitas maksimum. 3. save_checkpoint() : Membuat checkpoint dengan melakukan flush semua buffer ke disk, mencatat transaksi aktif, dan menyimpan checkpoint info untuk recovery point. 4. recover(criteria: RecoverCriteria) : Melakukan recovery dengan UNDO operasi secara backward berdasarkan transaction_id. Membaca semua log dari disk, filter berdasarkan kriteria, lalu rollback operasi dari yang terbaru ke awal. 5. commit_transaction(transaction_id: integer) : Fungsi ini memfinalisasi transaksi dengan menulis status COMMIT ke dalam log untuk memastikan semua perubahan data tersimpan secara permanen. 6. abort_transaction(transaction_id: integer) : Fungsi ini membatalkan transaksi yang sedang berjalan dengan menulis status ABORT ke log dan mengembalikan data ke kondisi sebelum transaksi dimulai (rollback)
--	---	--

	<pre> CHECKPOINT_INTERVAL_SECONDS = 300; void checkpoint_worker(); operation_to_string(Operation op); std::vector<Condition> row_to_conditions(const Row& row, const std::string& table_name); std::any string_to_any(const std::string& str); void write_string(std::ofstream& out, const std::string& str); std::string read_string(std::ifstream& in); void write_any(std::ofstream& out, const std::any& val); std::any read_any(std::ifstream& in); void write_columns(std::ofstream& out, const std::map<std::string, std::any>& columns); std::map<std::string, std::any> read_columns(std::ifstream& in); void write_row(std::ofstream& out, const Row& row); Row read_row(std::ifstream& in); void write_schema(std::ofstream& out, const TableSchema& schema); TableSchema read_schema(std::ifstream& in); LogEntry read_log_from_file(std::ifstream& in); </pre>	7. prepare_ddl_operation(table_name: string, op: Operation) : Fungsi ini menyiapkan dan mencatat operasi perubahan struktur database (seperti membuat atau menghapus tabel) ke dalam log untuk menjamin konsistensi skema saat pemulihan. 8. checkpoint_worker() : Fungsi ini berjalan secara otomatis di latar belakang (background thread) untuk melakukan proses checkpoint secara berkala guna menyinkronkan data dari memori ke disk. 9. operation_to_string(op: Operation) : Mengkonversi enum Operation menjadi string (BEGIN/COMMIT/ABORT/INSERT/UPDATE/DELETE/CHECKPOINT). 10. row_to_conditions(const Row& row, const std::string& table_name) : Mencari identifying condition (primary key) dari sebuah row pada tabel 11. string_to_any(const std::string& str) : Mengkonversi string menjadi std::any tipe data yang diinferensi. 12. write_string(std::ofstream& out, const std::string& str) : Menulis data string dalam bentuk binary ke file. 13. read_string(std::ifstream& in) : Membaca data string dalam bentuk binary dari file. 14. write_columns(std::ofstream& out, const std::map<std::string, std::any>& columns) : Menulis
--	--	--

	<pre> void write_log_to_file(std::ofstream& out, const LogEntry& entry); std::vector<LogEntry> read_all_logs(const std::string& file_path); std::string any_to_string(const std::any& val); std::string row_to_string(const Row& row); std::string schema_to_string(const TableSchema& schema); std::string sanitize_for_log(std::string input); void write_log_to_text_file(std::ofstr eam& out, const LogEntry& entry); std::set<int> parse_checkpoint_list(const std::string& query) bool undo_operation(const LogEntry& entry); bool redo_operation(const LogEntry& entry); void recover_from_crash(); void flush_buffer(); </pre>	data map columns dalam bentuk binary ke file. 15. read_columns(std::ifstream& in) : Membaca data map columns dalam bentuk binary dari file. 16. write_row(std::ofstream& out, const Row& row) : Menulis data Row dalam bentuk binary ke file. 17. read_row(std::ifstream& in) : Membaca data Row dalam bentuk binary dari file. 18. write_schema(std::ofstream& out, const TableSchema& schema) : Menulis data TableSchema dalam bentuk binary ke file. 19. TableSchema read_schema(std::ifstream& in) : Membaca data TableSchema dalam bentuk binary dari file. 20. read_log_from_file(std::ifstream& in) : Membaca sebuah data LogEntry dalam bentuk binary dari file. 21. write_log_to_file(std::ofstream& out, const LogEntry& entry) : Menulis sebuah data LogEntry dalam bentuk binary ke file. 22. read_all_logs(const std::string& file_path) : Membaca seluruh data LogEntry dalam bentuk binary dari file. 23. any_to_string(const std::any& val) : Mengkonversi `std::any` menjadi string untuk serialisasi (mendukung tipe int, float, dan string).
--	---	--

		24. row_to_string(const Row& row) : Melakukan serialisasi Row menjadi format string. 25. schema_to_string(const TableSchema& schema) : Melakukan serialisasi TableSchema menjadi format string. 26. sanitize_for_log(std::string input) : Fungsi helper untuk mengganti newline menjadi spasi pada string. 27. write_log_to_text_file(std::ofstream& out, const LogEntry& entry) : Menulis sebuah data LogEntry dalam bentuk plaintext ke file. 28. flush_buffer() : Menulis semua log entries di buffer ke disk file, lalu mengosongkan buffer untuk menghemat memory. 29. undo_operation(entry: LogEntry) : Melakukan UNDO operasi berdasarkan tipe: - INSERT → DELETE row baru (new_value) - DELETE → INSERT kembali row lama (old_value) - UPDATE → Restore ke nilai lama (old_value) 30. redo_operation(entry: LogEntry) : Melakukan REDO operasi berdasarkan tipe: - INSERT → INSERT row baru (new_value) - DELETE → Hapus kembali row lama (old_value) - UPDATE → Update ke nilai baru (new_value)
--	--	---

		31. recover_from_crash() : Melakukan recovery dari kegagalan sistem. Dimulai dengan proses mencari checkpoint terakhir, build undo_list. Kemudian, scan log setelah checkpoint untuk update status transaksi. Lalu, step REDO dan UNDO transaksi. 32. parse_checkpoint_list(const std::string& query) : Melakukan pembentukan undo_list dari log CHECKPOINT.
LogEntry	<pre> struct LogEntry { int log_id; int transaction_id; std::time_t timestamp; Operation operation; std::string table_name; Row old_value; Row new_value; std::string query; std::optional<TableSchema> dropped_schema; std::optional<TableSchema> created_schema; LogEntry() : log_id(-1), transaction_id(-1), timestamp(std::time(nullptr)) {} }; </pre>	Kelas ini merepresentasikan satu entri log yang disimpan saat suatu transaksi melakukan operasi.
CheckpointInfo	<pre> struct CheckpointInfo { int checkpoint_id; std::time_t timestamp; std::vector<int> active_transactions; CheckpointInfo() : checkpoint_id(-1), timestamp(std::time(nullptr)) {} }; </pre>	Merepresentasikan checkpoint yang disimpan untuk mempercepat proses recovery.
RecoverCriteria	<pre> struct RecoverCriteria { std::time_t timestamp; int transaction_id; }; </pre>	Kelas ini digunakan untuk menentukan bagaimana proses recovery dilakukan.

	<pre> bool use_timestamp; RecoverCriteria() : timestamp(0), transaction_id(-1), use_timestamp(false) {} }; </pre>	
Operation	<pre> enum class Operation { BEGIN, COMMIT, ABORT, UPDATE, INSERT, DELETE, CHECKPOINT, CREATE_TABLE, DROP_TABLE }; </pre>	Enumerasi untuk mendefinisikan jenis operasi yang dicatat dalam log
Row	<pre> struct Row { std::string table_name; std::map<std::string, std::any> columns; int row_id; Row() : row_id(-1) {} }; </pre>	Untuk merepresentasikan satu baris data dalam tabel

Pada milestone 1 ini, alur kerja integrasi difokuskan untuk mencapai kesepakatan desain dengan komponen lain, terutama penggunaan struktur data Row yang digunakan bagian Query Processor dan Concurrency Control Manager.

Pada milestone 2 ini, alur kerja difokuskan untuk mengimplementasikan fitur *transaction aborts recovery*. Beberapa method utama dikonstruksi untuk memastikan fitur ini dapat berjalan yaitu write_log() untuk menuliskan LogEntry ke buffer, save_checkpoint() untuk menambahkan LogEntry untuk checkpoint dan menyimpan informasi checkpoint, recover() untuk memastikan recovery dapat melakukan UNDO untuk transaksi yang di-abort, dan beberapa helper method tambahan seperti serialize_log() dan deserialize_log() untuk input/output LogEntry ke file dan sebaliknya, serta determine_operation_type() untuk penentuan operation type dari query string.

Pada milestone 3 ini, alur kerja difokuskan untuk mengimplementasikan fitur *recovery* akibat *system failure*. Beberapa method utama dikonstruksi untuk mengimplementasikan fitur ini antara lain `redo_operation()` yang melakukan operasi redo berdasarkan log entry dan `recover_from_crash()` yang melakukan serangkaian proses *recovery* akibat *system failure*. Juga dilakukan penambahan implementasi pada fungsi `undo_operation()` untuk menangani kasus INSERT, DELETE, UPDATE. Juga dilakukan perubahan minor pada fungsi `write_log()` untuk menangani pengambilan nilai `old_value`, `new_value`, dan `table_name` dari `ExecutionResult`. Terakhir, juga dilakukan implementasi penyimpanan log dengan format binary.

Pada milestone 4 ini, alur kerja difokuskan untuk mengintegrasikan fitur Failure Recovery Manager dengan Storage Manager, utamanya perihal manajemen buffer. Beberapa method yang dikonstruksi pada milestone 4 ini adalah `commit_transaction()` untuk memfinalisasi transaksi dengan commit, `abort_transaction()` untuk membatalkan transaksi yang sedang berjalan dengan abort, dan beberapa fungsi helper seperti `prepare_ddl_operation()`, `write_schema()`, `read_schema()`.

Part III. Experiment

1. Query Processor

a. Unit Testing *begin_transaction()*

Description	Menguji kemampuan Query Processor untuk memulai transaksi baru dan menghasilkan transaction ID yang unik.
Goal	Memastikan sistem dapat membuat transaksi baru dengan ID yang valid dan setiap transaksi baru mendapat ID yang lebih besar dari transaksi sebelumnya.
Method	1. Membuat instance QueryProcessor 2. Memulai transaksi pertama dengan <code>begin_transaction()</code> 3. Memulai transaksi kedua dengan <code>begin_transaction()</code> 4. Memvalidasi bahwa ID kedua lebih besar dari ID pertama
Success Criterion	- Transaction ID pertama ≥ 0 - Transaction ID kedua $>$ Transaction ID pertama
Input	Pemanggilan fungsi <code>begin_transaction()</code>
Expected Output	- <code>txn_id_1</code>: ≥ 0 - <code>txn_id_2</code>: $>$ <code>txn_id_1</code>
Results	SUCCESS <pre>=== TEST 2: Begin Transaction === QP: Query Processor initialized CCM: Transaksi 2 dimulai dengan timestamp 2 QP: Transaction 2 started FRM: Menulis log untuk query: BEGIN TRANSACTION (stub)... FRM: Transaksi 2 dimulai. FRM: Log ID 3 untuk T2 (BEGIN TRANSACTION) ditambahkan ke buffer. Op: BEGIN Transaction started with ID: 2 CCM: Transaksi 3 dimulai dengan timestamp 3 QP: Transaction 3 started FRM: Menulis log untuk query: BEGIN TRANSACTION (stub)... FRM: Transaksi 3 dimulai. FRM: Log ID 4 untuk T3 (BEGIN TRANSACTION) ditambahkan ke buffer. Op: BEGIN Second transaction started with ID: 3 [PASS] Begin Transaction test</pre>

b. Unit Testing *apply_limit()*

Description	Menguji fungsi LIMIT untuk membatasi jumlah baris yang dikembalikan dari hasil query.
-------------	---

Goal	Memverifikasi bahwa fungsi apply_limit dapat membatasi hasil query dengan benar sesuai dengan nilai limit yang diberikan.																																	
Method	1. Insert 10 data mahasiswa ke dalam tabel Student 2. Membaca semua data mahasiswa 3. Menerapkan LIMIT 5 menggunakan apply_limit() 4. Memvalidasi jumlah baris hasil																																	
Success Criterion	Jumlah baris hasil = 5																																	
Input	LIMIT value: 5 Data Tabel Students: <table><thead><tr><th>FullName</th><th>GPA</th><th>StudentID</th></tr></thead><tbody><tr><td>Alice</td><td>3.800</td><td>1</td></tr><tr><td>Bob</td><td>3.000</td><td>2</td></tr><tr><td>Charlie</td><td>3.500</td><td>3</td></tr><tr><td>Diana</td><td>3.500</td><td>4</td></tr><tr><td>Alice</td><td>2.500</td><td>5</td></tr><tr><td>Frank</td><td>3.700</td><td>6</td></tr><tr><td>Grace</td><td>3.400</td><td>7</td></tr><tr><td>Henry</td><td>2.800</td><td>8</td></tr><tr><td>Ivy</td><td>3.600</td><td>9</td></tr><tr><td>Alice</td><td>3.100</td><td>10</td></tr></tbody></table> 10 row(s) in set	FullName	GPA	StudentID	Alice	3.800	1	Bob	3.000	2	Charlie	3.500	3	Diana	3.500	4	Alice	2.500	5	Frank	3.700	6	Grace	3.400	7	Henry	2.800	8	Ivy	3.600	9	Alice	3.100	10
FullName	GPA	StudentID																																
Alice	3.800	1																																
Bob	3.000	2																																
Charlie	3.500	3																																
Diana	3.500	4																																
Alice	2.500	5																																
Frank	3.700	6																																
Grace	3.400	7																																
Henry	2.800	8																																
Ivy	3.600	9																																
Alice	3.100	10																																
Expected Output	- rows_count: 5 - Data: 5 baris pertama dari tabel Student																																	
Results	SUCCESS After LIMIT 5: <table><thead><tr><th>FullName</th><th>GPA</th><th>StudentID</th></tr></thead><tbody><tr><td>Alice</td><td>3.800</td><td>1</td></tr><tr><td>Bob</td><td>3.000</td><td>2</td></tr><tr><td>Charlie</td><td>3.500</td><td>3</td></tr><tr><td>Diana</td><td>3.500</td><td>4</td></tr><tr><td>Alice</td><td>2.500</td><td>5</td></tr></tbody></table> 5 row(s) in set [PASS] SELECT with LIMIT test	FullName	GPA	StudentID	Alice	3.800	1	Bob	3.000	2	Charlie	3.500	3	Diana	3.500	4	Alice	2.500	5															
FullName	GPA	StudentID																																
Alice	3.800	1																																
Bob	3.000	2																																
Charlie	3.500	3																																
Diana	3.500	4																																
Alice	2.500	5																																

c. Unit Testing `apply_order_by()`

Description	Menguji fungsi ORDER BY untuk mengurutkan
-------------	---

	data berdasarkan kolom string secara ascending (A-Z).
Goal	Memverifikasi bahwa fungsi <code>apply_order_by</code> dapat mengurutkan data string dengan benar dari A ke Z.
Method	1. Insert 10 data mahasiswa dengan berbagai nama 2. Membaca semua data mahasiswa 3. Menerapkan ORDER BY FullName ASC 4. Memvalidasi urutan hasil
Success Criterion	- Baris pertama memiliki FullName = "Alice" (A) - Baris terakhir memiliki FullName = "Ivy" (I)
Input	ORDER BY FullName ASC Data Tabel Students: <pre>Before ORDER BY: +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.800 1 Bob 3.000 2 Charlie 3.500 3 Diana 3.500 4 Alice 2.500 5 Frank 3.700 6 Grace 3.400 7 Henry 2.800 8 Ivy 3.600 9 Alice 3.100 10 +-----+-----+-----+ 10 row(s) in set</pre>
Expected Output	Urutan: Alice, Alice, Alice, Bob, Charlie, Diana, Frank, Grace, Henry, Ivy
Results	SUCCESS

	<pre> After ORDER BY FullName ASC: +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.800 1 Alice 2.500 5 Alice 3.100 10 Bob 3.000 2 Charlie 3.500 3 Diana 3.500 4 Frank 3.700 6 Grace 3.400 7 Henry 2.800 8 Ivy 3.600 9 +-----+-----+-----+ 10 row(s) in set [PASS] ORDER BY String ASC test </pre>
--	---

d. Unit Testing *apply_where_clause()*

Description	Menguji klausa WHERE dengan multiple kondisi yang digabungkan menggunakan operator AND.
Goal	Memverifikasi bahwa sistem dapat menangani multiple kondisi WHERE dengan benar dan mengembalikan hanya baris yang memenuhi semua kondisi.
Method	1. Insert 10 data mahasiswa 2. Buat Conditions WHERE GPA >= 3.5 AND StudentID > 3 AND FullName <> 'Diana' 3. Memvalidasi jumlah baris untuk setiap kombinasi kondisi
Success Criterion	2 baris (Frank ID=6 GPA=3.7, Ivy ID=9 GPA=3.6)
Input	Data Tabel Students: <pre> +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.800 1 Bob 3.000 2 Charlie 3.500 3 Diana 3.500 4 Alice 2.500 5 Frank 3.700 6 Grace 3.400 7 Henry 2.800 8 Ivy 3.600 9 Alice 3.100 10 +-----+-----+-----+ 10 row(s) in set </pre>
Expected Output	Sesuai dengan Success Criterion

Results	SUCCESS <pre> +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Frank 3.700 6 Ivy 3.600 9 +-----+-----+-----+ 2 row(s) in set [PASS] WHERE Multiple Conditions test </pre>
---------	--

e. Unit Testing *cartesian product*

Description	Menguji operasi Cartesian Product antara dua tabel tanpa kondisi join.
Goal	Memverifikasi bahwa sistem dapat menghasilkan Cartesian Product dengan benar, menghasilkan semua kombinasi baris dari kedua tabel.
Method	1. Insert 5 data mahasiswa 2. Insert 4 data course 3. Melakukan join tanpa kondisi 4. Memvalidasi jumlah baris hasil
Success Criterion	- Jumlah baris hasil = 20 (5 students × 4 courses) - Setiap baris memiliki kolom dari kedua tabel
Input	Data Tabel Courses: <pre> +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.800 1 Bob 3.000 2 Charlie 3.500 3 Diana 3.500 4 Alice 2.500 5 +-----+-----+-----+ 5 row(s) in set </pre> Data Tabel Students: <pre> Courses: +-----+-----+-----+ CourseID CourseName Year +-----+-----+-----+ 101 Database 2023 102 Algorithms 2024 103 Networks 2023 104 Operating Systems 2024 +-----+-----+-----+ 4 row(s) in set </pre>
Expected Output	20 baris hasil dengan semua kombinasi student dan course

Results

SUCCESS

Cartesian Product (5 students x 4 courses = 20 rows):

CourseID	CourseName	FullName	GPA	StudentID	Year
101	Database	Alice	3.800	1	2023
102	Algorithms	Alice	3.800	1	2024
103	Networks	Alice	3.800	1	2023
104	Operating Systems	Alice	3.800	1	2024
101	Database	Bob	3.000	2	2023
102	Algorithms	Bob	3.000	2	2024
103	Networks	Bob	3.000	2	2023
104	Operating Systems	Bob	3.000	2	2024
101	Database	Charlie	3.500	3	2023
102	Algorithms	Charlie	3.500	3	2024
103	Networks	Charlie	3.500	3	2023
104	Operating Systems	Charlie	3.500	3	2024
101	Database	Diana	3.500	4	2023
102	Algorithms	Diana	3.500	4	2024
103	Networks	Diana	3.500	4	2023
104	Operating Systems	Diana	3.500	4	2024
101	Database	Alice	2.500	5	2023
102	Algorithms	Alice	2.500	5	2024
103	Networks	Alice	2.500	5	2023
104	Operating Systems	Alice	2.500	5	2024

20 row(s) in set

[PASS] Cartesian Product test

f. Unit Testing `execute_join()` dengan JOIN ON

Description	Menguji operasi INNER JOIN dengan kondisi ON untuk menggabungkan dua tabel berdasarkan kolom yang sama.
Goal	Memverifikasi bahwa sistem dapat melakukan INNER JOIN dengan kondisi yang ditentukan dan hanya mengembalikan baris yang cocok.
Method	1. Insert 10 data mahasiswa 2. Insert 6 data course dengan Year yang sesuai dengan beberapa StudentID 3. Melakukan JOIN ON StudentID = Year 4. Memvalidasi jumlah dan isi hasil
Success Criterion	- Jumlah baris hasil = 5 - Hasil berisi: Alice+DB(1), Bob+Algo(2), Alice+Networks(5), Grace+OS(7), Alice+ML(10)
Input	Data Tabel Courses:

	<pre> Students: +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.800 1 Bob 3.000 2 Charlie 3.500 3 Diana 3.500 4 Alice 2.500 5 Frank 3.700 6 Grace 3.400 7 Henry 2.800 8 Ivy 3.600 9 Alice 3.100 10 +-----+-----+-----+ 10 row(s) in set </pre> Data Tabel Students: <pre> Courses: +-----+-----+-----+ CourseID CourseName Year +-----+-----+-----+ 101 Database 1 102 Algorithms 2 103 Networks 5 104 Operating Systems 7 105 Machine Learning 10 106 Distributed Systems 15 +-----+-----+-----+ 6 row(s) in set </pre>
Expected Output	5 baris dengan pasangan student-course yang sesuai. <pre> +-----+-----+-----+-----+-----+-----+ StudentID FullName GPA CourseID Year CourseName +-----+-----+-----+-----+-----+-----+ 1 Alice 3.8 101 1 Database 2 Bob 3.0 102 2 Algorithms 5 Alice 2.5 103 5 Networks 7 Grace 3.4 104 7 Operating Systems 10 Alice 3.1 105 10 Machine Learning +-----+-----+-----+-----+-----+-----+ </pre>
Results	SUCCESS <pre> QP: JOIN result: 5 rows JOIN ON StudentID = Year: +-----+-----+-----+-----+-----+-----+ CourseID CourseName FullName GPA StudentID Year +-----+-----+-----+-----+-----+-----+ 101 Database Alice 3.800 1 1 102 Algorithms Bob 3.000 2 2 103 Networks Alice 2.500 5 5 104 Operating Systems Grace 3.400 7 7 105 Machine Learning Alice 3.100 10 10 +-----+-----+-----+-----+-----+-----+ 5 row(s) in set [PASS] JOIN ON test </pre>

g. Unit Testing `execute_join()` dengan NATURAL JOIN

Description	Menguji operasi NATURAL JOIN yang secara otomatis melakukan join berdasarkan kolom
-------------	--

	dengan nama yang sama di kedua tabel.
Goal	Memverifikasi bahwa fungsi <code>execute_join</code> dapat melakukan NATURAL JOIN dengan benar, mencocokkan baris berdasarkan kolom yang sama.
Method	1. Membuat left table dengan kolom: ID, Name (10 baris) 2. Membuat right table dengan kolom: ID, Score (8 baris dengan beberapa ID tidak ada di left table) 3. Melakukan NATURAL JOIN 4. Memvalidasi jumlah dan struktur hasil
Success Criterion	- Jumlah baris hasil = 6 (ID yang cocok: 1, 2, 4, 5, 7, 10) - Hasil memiliki kolom: ID, Name, Score - Hanya baris dengan ID yang ada di kedua tabel yang dikembalikan
Input	Left Table: <pre> +-----+ ID Name +-----+ 1 Alice 2 Bob 3 Charlie 4 Diana 5 Eve 6 Frank 7 Grace 8 Henry 9 Ivy 10 Jack +-----+ 10 row(s) in set </pre> Right Table: <pre> +-----+ ID Score +-----+ 1 95 2 87 4 90 5 78 7 92 10 85 15 88 20 91 +-----+ 8 row(s) in set </pre>
Expected Output	Sesuai dengan Success Criterion

Results	SUCCESS <pre> QP: NATURAL JOIN result: 6 rows NATURAL JOIN (on ID): +-----+-----+-----+ ID Name Score +-----+-----+-----+ 1 Alice 95 2 Bob 87 4 Diana 90 5 Eve 78 7 Grace 92 10 Jack 85 +-----+-----+-----+ 6 row(s) in set [PASS] NATURAL JOIN test </pre>
---------	---

h. Unit Testing `execute_select()`

Description	Menguji eksekusi query SELECT dengan kombinasi klausa WHERE, ORDER BY, dan LIMIT menggunakan fungsi <code>execute_select</code> .
Goal	Memverifikasi bahwa fungsi <code>execute_select</code> dapat mengeksekusi query kompleks dengan multiple klausa dalam urutan yang benar.
Method	1. Insert 10 data mahasiswa 2. Membuat ParsedQuery untuk <code>SELECT * FROM Student WHERE GPA > 3.0 ORDER BY GPA DESC LIMIT 5</code> 3. Mengeksekusi query dengan <code>execute_select</code> 4. Memvalidasi jumlah, filter, dan urutan baris hasil
Success Criterion	- Jumlah baris hasil = 5 - Baris pertama memiliki GPA ≈ 3.8 - Hasil diurutkan descending - Semua GPA > 3.0
Input	Data Tabel Students:

	<pre> +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.800 1 Bob 3.000 2 Charlie 3.500 3 Diana 3.500 4 Alice 2.500 5 Frank 3.700 6 Grace 3.400 7 Henry 2.800 8 Ivy 3.600 9 Alice 3.100 10 +-----+-----+-----+ 10 row(s) in set [PASS] Execute SELECT Combined test </pre>
Expected Output	5 baris dengan GPA > 3.0 dan diurutkan descending <pre> +-----+-----+-----+ StudentID FullName GPA +-----+-----+-----+ 1 Alice 3.8 6 Frank 3.7 9 Ivy 3.6 3 Charlie 3.5 4 Diana 3.5 +-----+-----+-----+ </pre>
Results	SUCCESS <pre> +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.800 1 Frank 3.700 6 Ivy 3.600 9 Charlie 3.500 3 Diana 3.500 4 +-----+-----+-----+ 5 row(s) in set [PASS] Execute SELECT Combined test </pre>

i. Unit Testing UPDATE

Description	Menguji operasi UPDATE untuk mengubah nilai kolom pada multiple baris data sekaligus.
Goal	Memverifikasi bahwa fungsi execute_update dapat mengupdate multiple baris dengan benar berdasarkan kondisi WHERE.
Method	1. Insert 10 data mahasiswa 2. Mengeksekusi UPDATE Student SET GPA = 3.9 WHERE GPA > 3.5 3. Memvalidasi jumlah baris yang terpengaruh 4. Membaca data untuk memverifikasi perubahan

Success Criterion	- Affected rows = 3 (Alice 3.8, Frank 3.7, Ivy 3.6) - Ketiga baris tersebut memiliki GPA = 3.9 setelah update
Input	Data Tabel Students: <pre>Before UPDATE: +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.800 1 Bob 3.000 2 Charlie 3.500 3 Diana 3.500 4 Alice 2.500 5 Frank 3.700 6 Grace 3.400 7 Henry 2.800 8 Ivy 3.600 9 Alice 3.100 10 +-----+-----+-----+ 10 row(s) in set</pre>
Expected Output	Sesuai dengan Success Criterion
Results	SUCCESS <pre>Affected rows: 3 After UPDATE: +-----+-----+-----+ FullName GPA StudentID +-----+-----+-----+ Alice 3.900 1 Bob 3.000 2 Charlie 3.500 3 Diana 3.500 4 Alice 2.500 5 Frank 3.900 6 Grace 3.400 7 Henry 2.800 8 Ivy 3.900 9 Alice 3.100 10 +-----+-----+-----+ 10 row(s) in set [PASS] UPDATE Multiple Rows test</pre>

2. Query Optimizer

Berikut adalah contoh unit testing untuk komponen Query Optimizer.

a. Unit Test Query Parser

Description	Menguji apakah parser mampu mengekstrak komponen SELECT, FROM, JOIN, dan WHERE secara benar
Goal	Memastikan seluruh bagian query terbaca lengkap dan terstruktur
Method	Menjalankan parse_query() pada query kompleks dan memeriksa hasil parsing
Success Criterion	Semua kolom, tabel, join pairs, dan kondisi WHERE terdeteksi dengan benar
Input	<pre> SELECT s.name, s.age, s.address, d.nama, d.size, a.name, a.type FROM student s JOIN dept d ON s.dept_id = d.id JOIN apt a ON a.id = s.apt_id WHERE s.age > 20 AND s.age < 60 AND d.size >= 10 AND d.location = 'Jakarta' AND a.name = 'bebek' AND a.active = 1; </pre>
Expected Output	Sesuai dengan success criterion
Results	SUCCESS <pre> SELECT columns: s.name;s.age;s.address;d.nama;d.size;a.name;a.type; FROM tables: student s;dept d;apt a; JOIN pairs: Join 1: s.dept_id = d.id Join 2: a.id = s.apt_id WHERE conditions: Condition 1: s.age > [value] Condition 2: s.age < [value] Condition 3: d.size >= [value] Condition 4: d.location = [value] Condition 5: a.name = [value] Condition 6: a.active = [value] </pre>

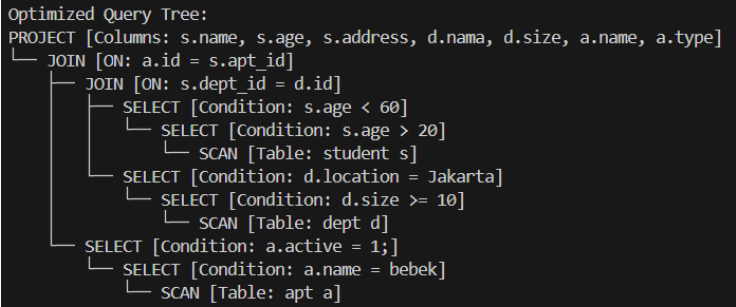
b. Unit Test Query Tree

Description	Menguji apakah Query Tree awal terbentuk dengan struktur PROJECT → SELECT → JOIN → SCAN
Goal	Memastikan tree hasil parsing mencerminkan urutan eksekusi logis query

Method	Mencetak tree dari parsed_query.query_tree dan memeriksa struktur nya
Success Criterion	Node SELECT muncul sesuai kondisi, JOIN membentuk tree biner, dan SCAN sesuai tabel
Input	SELECT s.name, s.age, s.address, d.nama, d.size, a.name, a.type FROM student s JOIN dept d ON s.dept_id = d.id JOIN apt a ON a.id = s.apt_id WHERE s.age > 20 AND s.age < 60 AND d.size >= 10 AND d.location = 'Jakarta' AND a.name = 'bebek' AND a.active = 1;
Expected Output	Sesuai dengan success criterion
Results	SUCCESS <pre> Initial Query Tree: PROJECT [Columns: s.name, s.age, s.address, d.nama, d.size, a.name, a.type] ├─ SELECT [Condition: a.active = 1;] │ └─ SELECT [Condition: a.name = bebek] │ └─ SELECT [Condition: d.location = Jakarta] │ └─ SELECT [Condition: d.size >= 10] │ └─ SELECT [Condition: s.age > 20] │ └─ JOIN [ON: a.id = s.apt_id] │ └─ JOIN [ON: s.dept_id = d.id] │ └─ SCAN [Table: student s] │ └─ SCAN [Table: dept d] │ └─ SCAN [Table: apt a] </pre>

c. Unit Test Optimizer

Description	Menguji apakah optimize_tree() melakukan splitting conjunction, reorder selections, pushdown selection, dan projection redundant
Goal	Memastikan optimized tree lebih efisien tanpa mengubah semantics query
Method	Memanggil optimize_query(parsed_query) dan memeriksa struktur tree hasil optimasi
Success Criterion	SELECT tersplit, SELECT dipush-down ke tabel masing-masing, JOIN tersusun rapi, dan projection redundan dihapus
Input	SELECT s.name, s.age, s.address, d.nama, d.size,

	a.name, a.type FROM student s JOIN dept d ON s.dept_id = d.id JOIN apt a ON a.id = s.apt_id WHERE s.age > 20 AND s.age < 60 AND d.size >= 10 AND d.location = 'Jakarta' AND a.name = 'bebek' AND a.active = 1;
Expected Output	Sesuai dengan success criterion
Results	SUCCESS 

3. Concurrency Control Manager

Berikut adalah contoh unit testing untuk komponen Concurrency Control Manager.

a. Unit Test Singleton Pattern

Description	Menguji pattern Singleton pada ConcurrencyControlManager untuk memastikan hanya ada satu instance yang dibuat.
Goal	Memverifikasi bahwa pemanggilan get_instance() selalu mengembalikan pointer instance yang sama.
Method	Memanggil get_instance() dua kali dan membandingkan alamat memori yang dikembalikan.
Success Criterion	Kedua instance memiliki alamat memori yang sama.
Input	Tidak ada input khusus.

Expected Output	Alamat instance c1 sama dengan c2.
Results	SUCCESS <pre> --- TEST 1: Singleton Pattern --- CCM: Protokol Timestamp Ordering diinisialisasi CCM: Beralih ke protokol concurrency control timestamp CCM: Instance Singleton dibuat [PASS] Singleton instance konsisten --- TEST 1.1: Singleton Switch Algorithm --- CCM: Beralih ke protokol concurrency control twophaselocking [PASS] Singleton Switch konsisten CCM: Protokol Timestamp Ordering diinisialisasi CCM: Beralih ke protokol concurrency control timestamp </pre>

b. Unit Test Thread Safety Singleton

Description	Menguji thread-safety dari Singleton ConcurrencyControlManager.
Goal	Memastikan semua thread mendapat instance yang sama meskipun dipanggil secara bersamaan.
Method	Membuat 10 thread, masing-masing memanggil get_instance() dan menyimpan pointer. Kemudian dibandingkan semua pointer.
Success Criterion	Semua thread mendapatkan pointer yang sama.
Input	10 thread
Expected Output	Semua thread memiliki pointer instance yang sama.
Results	SUCCESS <pre> --- TEST 2: Thread Safety Singleton --- [PASS] Semua thread mendapat instance yang sama </pre>

c. Unit Test Basic Transaction Lifecycle

Description	Menguji siklus hidup transaksi dasar menggunakan timestamp CC.
Goal	Memastikan transaksi dapat mulai, membaca, menulis, dan diakhiri tanpa error.
Method	Memulai transaksi, melakukan log objek, validate READ & WRITE, lalu commit.

Success Criterion	READ & WRITE diizinkan dan transaksi berhasil diakhiri.
Input	Row dengan row_id=1 dan data_col="test_table"
Expected Output	READ & WRITE return allowed=true, transaksi selesai tanpa exception.
Results	SUCCESS <pre> --- TEST 3: Basic Transaction Lifecycle --- CCM: Transaksi 1 dimulai dengan timestamp 1 CCM: Transaksi 1 mencatat akses ke objek (hash: 3586352716876597368) pada tabel TestTable CCM: Transaksi 1 diizinkan READ pada objek (hash: 3586352716876597368), read_ts diupdate ke 1 CCM: Transaksi 1 diizinkan WRITE pada objek (hash: 3586352716876597368), write_ts diupdate ke 1 CCM: Transaksi 1 memasuki state PARTIALLY_COMMITTED CCM: Transaksi 1 COMMITTED dengan sukses CCM: Cleanup resource transaksi 1 [PASS] Transaction lifecycle berhasil </pre>

d. Unit Test Time Stamp Read after Write

Description	Menguji deteksi konflik read-after-write pada protokol timestamp.
Goal	Memastikan read oleh transaksi lebih tua gagal jika transaksi lebih muda sudah menulis.
Method	Buat 3 transaksi, lakukan WRITE oleh T1, WRITE oleh T3, kemudian READ oleh T2.
Success Criterion	READ oleh T2 gagal (allowed=false).
Input	Row row_id=100
Expected Output	T2 READ return allowed=false
Results	SUCCESS <pre> --- TEST 4: Read After Write Conflict --- CCM: Transaksi 2 dimulai dengan timestamp 2 CCM: Transaksi 2 mencatat akses ke objek (hash: 3586352716876597269) pada tabel TestTable CCM: Transaksi 2 diizinkan WRITE pada objek (hash: 3586352716876597269), write_ts diupdate ke 2 CCM: Transaksi 2 memasuki state PARTIALLY_COMMITTED CCM: Transaksi 2 COMMITTED dengan sukses CCM: Cleanup resource transaksi 2 CCM: Transaksi 3 dimulai dengan timestamp 3 CCM: Transaksi 3 mencatat akses ke objek (hash: 3586352716876597269) pada tabel TestTable CCM: Transaksi 4 dimulai dengan timestamp 4 CCM: Transaksi 4 mencatat akses ke objek (hash: 3586352716876597269) pada tabel TestTable CCM: Transaksi 4 diizinkan WRITE pada objek (hash: 3586352716876597269), write_ts diupdate ke 4 CCM: Transaksi 4 memasuki state PARTIALLY_COMMITTED CCM: Transaksi 4 COMMITTED dengan sukses CCM: Cleanup resource transaksi 4 CCM: Transaksi 3 GAGAL karena konflik timestamp pada READ (TS=3 < write_ts=4) CCM: Transaksi 3 melakukan rollback (FAILED -> ABORTED) CCM: Transaksi 3 dihentikan (rollback selesai) CCM: Cleanup resource transaksi 3 [PASS] Timestamp read conflict terdeteksi </pre>

e. Unit Test Timestamp Write After Read Conflict

Description	Menguji deteksi konflik write-after-read pada protokol timestamp.
Goal	Memastikan WRITE oleh transaksi lebih tua gagal jika transaksi lebih muda sudah READ.
Method	Buat transaksi TA baca, TB tulis setelah TA selesai, TC baca, lalu TB menulis.
Success Criterion	WRITE oleh TB gagal (allowed=false).
Input	Row row_id=200
Expected Output	TB WRITE return allowed=false
Results	SUCCESS <pre> --- TEST 5: Write After Read Conflict --- CCM: Transaksi 5 dimulai dengan timestamp 5 CCM: Transaksi 5 mencatat akses ke objek (hash: 3586352716876597425) pada tabel TestTable CCM: Transaksi 5 diizinkan READ pada objek (hash: 3586352716876597425), read_ts diupdate ke 5 CCM: Transaksi 6 dimulai dengan timestamp 6 CCM: Transaksi 5 memasuki state PARTIALLY_COMMITTED CCM: Transaksi 5 COMMITTED dengan sukses CCM: Cleanup resource transaksi 5 CCM: Transaksi 7 dimulai dengan timestamp 7 CCM: Transaksi 7 mencatat akses ke objek (hash: 3586352716876597425) pada tabel TestTable CCM: Transaksi 7 diizinkan READ pada objek (hash: 3586352716876597425), read_ts diupdate ke 7 CCM: Transaksi 7 memasuki state PARTIALLY_COMMITTED CCM: Transaksi 7 COMMITTED dengan sukses CCM: Cleanup resource transaksi 7 CCM: Transaksi 6 mencatat akses ke objek (hash: 3586352716876597425) pada tabel TestTable CCM: Transaksi 6 GAGAL karena konflik timestamp pada WRITE (TS=6, read_ts=7, write_ts=0) [PASS] Timestamp write conflict terdeteksi CCM: Transaksi 6 melakukan rollback (FAILED -> ABORTED) CCM: Transaksi 6 dihentikan (rollback selesai) CCM: Cleanup resource transaksi 6 </pre>

f. Unit Test Concurrent Transactions

Description	Menguji concurrency control di bawah transaksi paralel.
Goal	Memastikan transaksi paralel tetap mengikuti protokol CC tanpa crash.
Method	5 thread masing-masing melakukan 10 transaksi READ & WRITE secara bersamaan.
Success Criterion	Semua transaksi selesai, sebagian READ & WRITE berhasil sesuai aturan CC.
Input	5 thread, 10 operasi per thread, Row row_id=c
Expected Output	Semua thread selesai, total operasi OK + fail = N*OPS

Results	SUCCESS <pre> --- TEST 6: Concurrent Transactions --- COM: Transaksi 8 dimulai dengan timestamp 8 COM: Transaksi 8 mencatat akses ke objek (hash: 3586352716876597368) pada tabel TestTable COM: Transaksi 8 diizinkan READ pada objek (hash: 3586352716876597368), read_ts diupdate ke 8 COM: Transaksi 8 diizinkan WRITE pada objek (hash: 3586352716876597368), write_ts diupdate ke 8 COM: Transaksi 9 dimulai dengan timestamp 9 COM: Transaksi 9 mencatat akses ke objek (hash: 3586352716876597365) pada tabel TestTable COM: Transaksi 10 dimulai dengan timestamp 10 COM: Transaksi 10 mencatat akses ke objek (hash: 3586352716876597364) pada tabel TestTable COM: Transaksi 10 diizinkan READ pada objek (hash: 3586352716876597364), read_ts diupdate ke 10 COM: Transaksi 10 diizinkan WRITE pada objek (hash: 3586352716876597364), write_ts diupdate ke 10 COM: Transaksi 10 memasuki state PARTIALLY_COMMITTED COM: Transaksi 10 COMMITTED dengan sukses COM: Cleanup resource transaksi 10 COM: Cleanup resource transaksi 55 COM: Transaksi 57 dimulai dengan timestamp 57 COM: Transaksi 57 mencatat akses ke objek (hash: 3586352716876597364) pada tabel TestTable COM: Transaksi 57 diizinkan READ pada objek (hash: 3586352716876597364), read_ts diupdate ke 57 COM: Transaksi 57 diizinkan WRITE pada objek (hash: 3586352716876597364), write_ts diupdate ke 57 COM: Transaksi 57 memasuki state PARTIALLY_COMMITTED COM: Transaksi 57 COMMITTED dengan sukses COM: Cleanup resource transaksi 57 [PASS] Concurrent transactions berhasil </pre>
---------	---

g. Unit Test Invalid Transaction ID

Description	Menguji penanganan ID transaksi yang tidak valid.
Goal	Memastikan error ditangkap saat log_object dengan ID yang tidak ada.
Method	Panggil log_object dengan ID transaksi 999999.
Success Criterion	Exception ditangkap.
Input	Row row_id=999, transaction_id=999999
Expected Output	Exception ditangkap, test PASS
Results	SUCCESS <pre> --- TEST 7: Invalid Transaction ID --- [PASS] Invalid transaction ID terdeteksi </pre>

h. Unit Test Switch Algorithm

Description	Menguji switch antara algoritma timestamp dan invalid algorithm.
Goal	Memastikan switch ke timestamp berhasil dan switch ke algoritma tidak valid dilempar exception.
Method	Panggil switch_algorithm("timestamp") dan switch_algorithm("unsupported_algorithm").
Success Criterion	Timestamp berhasil, unsupported algorithm

	dilempar exception.
Input	Algoritma: "timestamp" / "unsupported_algorithm"
Expected Output	Timestamp: no error, Unsupported: exception thrown
Results	SUCCESS <pre> --- TEST 8: Switch Algorithm --- CCM: Protokol Timestamp Ordering diinisialisasi CCM: Beralih ke protokol concurrency control timestamp [PASS] Switch ke timestamp berhasil [PASS] Unsupported algorithm terdeteksi </pre>

i. Unit Test 2PL Concurrency General

Description	Menguji operasi dasar Two-Phase Locking: READ, WRITE, lock conflict, lock upgrade.
Goal	Memastikan protokol 2PL berjalan sesuai aturan lock.
Method	Simulasi proses <i>read records</i> , <i>shared locks</i> , <i>exclusive locks</i> , dan menunggu <i>lock</i> yang sedang dimiliki oleh transaksi lain
Success Criterion	Lock conflict ditolak, operasi lain berhasil sesuai aturan 2PL.
Input	Row1 id=1, Row2 id=2
Expected Output	T1 READ Row1: allowed, T2 READ Row1: allowed, T2 WRITE Row1: reject, T2 WRITE Row2: allowed, T3 READ Row2: reject
Results	SUCCESS

	<pre> --- TEST 9: 2PL Concurrency General --- CCM: Beralih ke protokol concurrency control twophaselocking 2PL: Transaksi 1 dimulai (fase GROWING). 2PL: Transaksi 1 mencoba memperoleh S lock pada record 4361635078269866268 2PL: Transaksi 1 memperoleh S lock pada record 4361635078269866268 Transaction 1 READ on Row1: Allowed: True, TID: 1 [PASS] T1 READ Row1 2PL: Transaksi 2 dimulai (fase GROWING). 2PL: Transaksi 2 mencoba memperoleh S lock pada record 4361635078269866268 2PL: Transaksi 2 memperoleh S lock pada record 4361635078269866268 Transaction 2 READ on Row1: Allowed: True, TID: 2 [PASS] T2 READ Row1 2PL: Transaksi 2 mencoba memperoleh X lock pada record 4361635078269866268 2PL: Transaksi 2 menunggu S lock pada record 4361635078269866268 yang dipegang oleh transaksi 1 2PL: Transaksi 2 masuk mode WAITING... Transaction 2 WRITE on Row1: Allowed: False, TID: 2 [PASS] T2 WRITE Row1 (expect reject) 2PL: Transaksi 2 mencoba memperoleh X lock pada record 4361635078269866271 2PL: Transaksi 2 memperoleh X lock pada record 4361635078269866271 Transaction 2 WRITE on Row2: Allowed: True, TID: 2 [PASS] T2 WRITE Row2 2PL: Transaksi 3 dimulai (fase GROWING). 2PL: Transaksi 3 mencoba memperoleh S lock pada record 4361635078269866271 2PL: Transaksi 3 menunggu X lock pada record 4361635078269866271 yang dipegang oleh transaksi 2 2PL: Transaksi 3 masuk mode WAITING... Transaction 3 READ on Row2: Allowed: False, TID: 3 [PASS] T3 READ Row2 (failed karena T2 memegang xlock) 2PL: Transaksi 1 berhasil commit (fase SHRINKING). 2PL: Transaksi 1 melepaskan S lock pada record 4361635078269866268 2PL: Transaksi 2 sedang menunggu, transaksi akan diabort. 2PL: Transaksi 2 melepaskan S lock pada record 4361635078269866268 2PL: Transaksi 2 melepaskan X lock pada record 4361635078269866271 2PL: Transaksi 3 sedang menunggu, transaksi akan diabort. </pre>
--	--

j. Unit Test 2PL Abort Scenarios

Description	Menguji perilaku transaksi yang commit (SHRINKING phase) dan mencoba lock baru.
Goal	Memastikan transaksi tidak dapat lock baru setelah commit. (abort)
Method	T1 READ Row1, T2 WRITE Row2, T1 commit, T1 READ Row2 (harus fail).
Success Criterion	T1 READ Row2 after commit = fail
Input	Row1 id=10, Row2 id=11
Expected Output	T1 READ Row2: allowed=false
Results	SUCCESS

	<pre> --- TEST 10: Abort Scenarios --- CCM: Beralih ke protokol concurrency control twophaselocking 2PL: Transaksi 1 dimulai (fase GROWING). 2PL: Transaksi 1 mencoba memperoleh S lock pada record 3586352716876597363 2PL: Transaksi 1 memperoleh S lock pada record 3586352716876597363 Transaction 1 READ on Row1: Allowed: True, TID: 1 [PASS] T1 READ Row1 (expect allowed) 2PL: Transaksi 2 dimulai (fase GROWING). 2PL: Transaksi 2 mencoba memperoleh X lock pada record 3586352716876597362 2PL: Transaksi 2 memperoleh X lock pada record 3586352716876597362 Transaction 2 WRITE on Row2: Allowed: True, TID: 2 [PASS] T2 WRITE Row2 (expect allowed) 2PL: Transaksi 1 berhasil commit (fase SHRINKING). 2PL: Transaksi 1 melepaskan S lock pada record 3586352716876597363 2PL: Transaksi 1 melepaskan semua kunci. 2PL: Transaksi 1 sudah selesai (COMMITTED), tidak dapat mengakses lebih lanjut Transaction 1 READ on Row2: Allowed: False, TID: 1 [PASS] T1 READ Row2 after commit (expect fail) 2PL: Transaksi 2 berhasil commit (fase SHRINKING). 2PL: Transaksi 2 melepaskan X lock pada record 3586352716876597362 2PL: Transaksi 2 melepaskan semua kunci. </pre>
--	---

k. Unit Test 2PL Deadlock Scenario

Description	Menguji deadlock handling dasar: transaksi menunggu X-lock tapi langsung abort jika conflict.
Goal	Memastikan deadlock ditangani dengan abort transaksi yang konflik.
Method	T1 read R1, T2 read R2, T1 write R2 (reject), T2 write R1 (<i>deadlock detected</i> → T2 aborted), T1 write R1 after T2 <i>aborted</i> (allow)
Success Criterion	Konflik menyebabkan <i>abort</i> dan transaksi yang tidak <i>aborted</i> tetap beroperasi.
Input	Row1 id=20, Row2 id=21
Expected Output	T1 di akhir dapat menulis R1 karena T2 otomatis <i>aborted</i> setelah terjadi <i>deadlock</i>
Results	SUCCESS

	<pre> --- TEST 11: Deadlock Scenarios --- CCM: Beralih ke protokol concurrency control twophaselocking 2PL: Transaksi 1 dimulai (fase GROWING). 2PL: Transaksi 1 mencoba memperoleh S lock pada record 4361635078269866249 2PL: Transaksi 1 memperoleh S lock pada record 4361635078269866249 Transaction 1 READ on Row1: Allowed: True, TID: 1 [PASS] T1 READ Row1 (expect allowed) 2PL: Transaksi 2 dimulai (fase GROWING). 2PL: Transaksi 2 mencoba memperoleh S lock pada record 4361635078269866248 2PL: Transaksi 2 memperoleh S lock pada record 4361635078269866248 Transaction 2 READ on Row2: Allowed: True, TID: 2 [PASS] T2 READ Row2 (expect allowed) 2PL: Transaksi 1 mencoba memperoleh X lock pada record 4361635078269866248 2PL: Transaksi 1 menunggu S lock pada record 4361635078269866248 yang dipegang oleh transaksi 2 2PL: Transaksi 1 masuk mode WAITING... Transaction 1 WRITE on Row2: Allowed: False, TID: 1 [PASS] T1 WRITE Row2 (expect fail due to conflict) 2PL: Transaksi 2 mencoba memperoleh X lock pada record 4361635078269866249 2PL: Deadlock terdeteksi antara transaksi 2 dan 1 2PL: Deadlock terdeteksi antara transaksi 2 dan 1 2PL: Transaksi 2 diabort (fase SHRINKING). 2PL: Transaksi 2 melepaskan S lock pada record 4361635078269866248 2PL: Transaksi 2 selesai diabort. Transaction 2 WRITE on Row1: Allowed: False, TID: 2 [PASS] T2 WRITE Row1 (expect fail due to abort) 2PL: Transaksi 1 mencoba memperoleh X lock pada record 4361635078269866249 2PL: Transaksi 1 mengupgrade S lock ke X lock pada record 4361635078269866249 2PL: Transaksi 1 memperoleh X lock pada record 4361635078269866249 Transaction 1 WRITE on Row1: Allowed: True, TID: 1 [PASS] T1 WRITE Row1 after T2 aborted (expect allowed) 2PL: Transaksi 1 sedang menunggu, transaksi akan diabort. 2PL: Transaksi 1 melepaskan X lock pada record 4361635078269866249 2PL: Transaksi 2 sudah diabort. </pre>
--	--

I. Unit Test 2PL with Multithreading

Description	Pengujian perilaku Two-Phase Locking (2PL) dalam situasi multithread sederhana: T1 memegang lock terlebih dahulu, T2 mencoba mengambil lock dan harus menunggu hingga T1 selesai.
Goal	Memastikan bahwa mekanisme 2PL menahan transaksi yang datang belakangan (T2) hingga lock dilepas oleh transaksi pertama (T1), serta memastikan tidak ada bypass atau conflict execution.
Method	1. T1 memulai transaksi, melakukan WRITE lock pada row 1, lalu menahan lock selama 2 detik. 2. Setelah 500 ms, T2 memulai transaksi dan memanggil validate_object via execute_with_retry. T2 harus masuk ke mode retry/wait karena lock masih dipegang T1. 3. Setelah T1 commit dan lock dilepas, T2 harus dapat memperoleh lock dan menyelesaikan transaksi.
Success Criterion	T2 tidak boleh mendapatkan lock sebelum T1 selesai commit. Tidak ada deadlock atau starvation.
Input	- Row: row1 (id=1, "Rebutan")

	- T1: WRITE lock, menjalankan selama 2 detik. - T2: WRITE lock, datang 500 ms setelah T1. - Algoritma aktif: Two-Phase Locking (2PL).
Expected Output	T2 berhasil mendapatkan <i>lock</i> dan menjalankan operasi setelah T1 <i>commit</i> dan melewati <i>lock</i> .
Results	<pre> [T1] commit. 2PL: Transaksi 1 berhasil commit (fase SHRINKING). 2PL: Transaksi 1 melepaskan X lock pada record 4361635078269866268 2PL: Transaksi 1 melepaskan semua kunci. isi dari wait-lock queue setelah release all locks: Record 4361635078269866268: 2 2PL: Transaksi 2 mencoba memperoleh X lock pada record 4361635078269866268 2PL: Transaksi 2 memperoleh X lock pada record 4361635078269866268 [T2] SUKSES mendapatkan Lock di percobaan ke-3 </pre>

m. Unit Test 2PL in regard of Queue Fairness

Description	Menguji apakah algoritma Two-Phase Locking (2PL) menerapkan fairness dalam antrian lock: T2 harus mendapatkan lock sebelum T3 karena T2 datang lebih dulu.
Goal	Memastikan bahwa lock diberikan berdasarkan urutan kedatangan (FIFO queue), bukan berdasarkan timing randomness atau starvation.
Method	T1 mengunci row 1 terlebih dahulu. lalu setelah 500 ms, T2 mencoba WRITE lock kemudian masuk antrian dan menunggu. Setelah 200 ms, T3 mencoba WRITE lock namun berada di belakang T2. Saat T1 commit, lock harus diberikan ke T2 dulu lalu ke T3.
Success Criterion	Pemberian <i>lock</i> setelah T1 <i>commit</i> harus diterima oleh T2 terlebih dahulu kemudian T3. Dengan demikian tidak terdapat <i>starvation</i> dan tidak terdapat transaksi yang <i>bypass</i> antrian.
Input	T1: WRITE lock, menjalankan 2 detik. T2: WRITE lock, datang 500 ms setelah T1. T3: WRITE lock, datang 700 ms setelah T1.
Expected Output	T3 tidak mendapatkan akses setelah T1 melepas <i>lock</i> karena urutannya harus T2 terlebih dahulu

Results	<pre> [T1] commit. 2PL: Transaksi 1 berhasil commit (fase SHRINKING). 2PL: Transaksi 1 melepaskan X lock pada record 4361635078269866268 2PL: Transaksi 1 melepaskan semua kunci. isi dari wait-lock queue setelah release all locks: Record 4361635078269866268: 2 3 2PL: Transaksi 3 mencoba memperoleh X lock pada record 4361635078269866268 2PL: Transaksi 3 masih dalam antrian untuk record 4361635078269866268 2PL: Transaksi 3 masuk mode WAITING... [T3] Gagal (Wait). Tidur 100ms... </pre>
---------	--

4. Storage Manager

Berikut adalah tiga contoh unit testing untuk komponen Storage Manager pada Milestone 3.

a. Unit Test Membuat Tabel Baru - Test 0

Description	Membuat tabel Student dan Course
Goal	Memastikan schema tabel berhasil terbuat
Method	Memanggil create_table(student_schema) Memanggil create_table(course_schema)
Success Criterion	Terbuat file Student.schema dan Course.schema serta tables.meta dengan benar
Input	<pre> TableSchema student_schema; student_schema.table_name = "Student"; student_schema.column_names = {"StudentID", "FullName", "GPA"}; student_schema.column_types = {DataType::INTEGER, DataType::VARCHAR, DataType::FLOAT}; student_schema.column_sizes = {0, 50, 0}; student_schema.primary_key = "StudentID"; student_schema.foreign_keys = {}; sm.create_table(student_schema); </pre>
Expected Output	<pre> DEBUG: sm_test_data/Student.schema size = 66 bytes DEBUG: sm_test_data/Course.schema size = 67 bytes Test 0 OK. </pre>

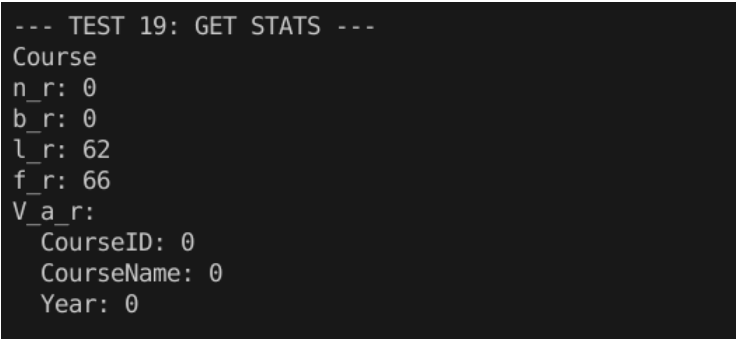
Results	SUCCESS <pre> --- TEST 0: CREATE TABLE --- DEBUG: sm_test_data/Student.schema size = 66 bytes DEBUG: sm_test_data/Course.schema size = 67 bytes Test 0 OK. </pre>
---------	--

b. Unit Test Drop Tabel - Test 17

Description	Menghapus tabel Student
Goal	Memverifikasi tabel student sudah terhapus beserta file-file yang bersangkutan
Method	Memanggil delete_table(student_schema) dengan student_schema adalah TableSchema yang telah dibuat pada test sebelumnya Kemudian melakukan query kepada tabel student
Success Criterion	Gagal membaca schema yang telah dihapus
Input	sm.delete_table(student_schema); rows_all = sm.read_block(read_all);
Expected Output	SM: Gagal membuka file schema SM: Error - File skema tidak ditemukan untuk tabel: Student
Results	SUCCESS <pre> --- TEST 17: DROP TABLE --- Dropping student table Can't get student schema error is expected: SM: Gagal membuka file schema SM: Error - File skema tidak ditemukan untuk tabel: Student --- Hasil Query (Total: 0) --- </pre>

c. Unit Test get_stats - Test 19

Description	Mendapatkan statistic dari semua tabel menggunakan fungsi get_stats.
Goal	Memastikan get_stats mengembalikan data yang benar
Method	Memanggil get_stats dan menampilkan data yang

	didapatkan ke terminal
Success Criterion	Mendapatkan semua statistik dari semua tabel yang ada
Input	sm.get_stats()
Expected Output	<pre>--- TEST 19: GET STATS --- Course n_r: 0 b_r: 0 l_r: 62 f_r: 66 V_a_r: CourseID: 0 CourseName: 0 Year: 0</pre> (Course memang kosong dan Student tidak ada karena di drop pada test sebelumnya)
Results	SUCCESS  <pre>--- TEST 19: GET STATS --- Course n_r: 0 b_r: 0 l_r: 62 f_r: 66 V_a_r: CourseID: 0 CourseName: 0 Year: 0</pre>

5. Failure Recovery

Berikut adalah beberapa unit testing untuk komponen Failure Recovery.

a. Unit Test Singleton Pattern

Description	Pengujian untuk memastikan FailureRecoveryManager mengimplementasikan Singleton Pattern dengan benar.
Goal	Memastikan bahwa hanya ada satu instance dari FailureRecoveryManager yang dibuat dan digunakan.

Method	Memanggil <code>FailureRecoveryManager::get_instance()</code> dua kali dan membandingkan alamat memori kedua instance tersebut. Jika alamat memori sama, maka Singleton Pattern berhasil diimplementasikan.
Success Criterion	Alamat memori dari kedua instance harus sama.
Input	Tidak ada input eksplisit. Test memanggil <code>get_instance()</code> dua kali.
Expected Output	Kedua instance memiliki alamat memori yang sama, dan test menampilkan "PASS: Singleton valid (Alamat Memori Sama)."
Results	SUCCESS <pre>[TEST] Checking Singleton Property... FRM: Konstruktor FailureRecoveryManager dipanggil (stub)... PASS: Singleton valid (Alamat Memori Sama).</pre>

b. Unit Test Write Log Persistence

Description	Pengujian untuk memastikan log yang ditulis melalui <code>write_log()</code> dapat disimpan secara persisten ke file.
Goal	Memastikan bahwa log transaksi (BEGIN, INSERT, dll) dan checkpoint dapat ditulis ke file <code>data/wal.log</code> dan dapat dibaca kembali.
Method	1. Membuat ExecutionResult untuk BEGIN TRANSACTION (T100). 2. Membuat ExecutionResult untuk INSERT query (T101). 3. Memanggil <code>write_log()</code> untuk kedua operasi. 4. Memanggil <code>save_checkpoint()</code> 5. Membaca file log dan memverifikasi bahwa log dan checkpoint tersimpan
Success Criterion	File <code>data/wal.log</code> harus berisi: - Log entry untuk BEGIN TRANSACTION - Log entry untuk INSERT INTO mahasiswa - Log entry untuk CHECKPOINT
Input	- ExecutionResult untuk BEGIN: <code>transaction_id=100, query="BEGIN</code>

	TRANSACTION", success=true - ExecutionResult untuk INSERT: transaction_id=101, query="INSERT INTO mahasiswa VALUES (1, 'Budi')", success=true
Expected Output	File log berisi 3 baris: 1. BEGIN log 2. INSERT log 3. CHECKPOINT log Format: LOG_ID TX_ID TIMESTAMP OP TABLE OLD_VAL NEW_VAL QUERY
Results	SUCCESS <pre>[TEST] Checking Write Log Persistence... FRM: Menulis log untuk query: BEGIN TRANSACTION (stub)... FRM: Transaksi 100 dimulai. FRM: Log ID 1 untuk T100 (BEGIN TRANSACTION) ditambahkan ke buffer. Op: BEGIN FRM: Menulis log untuk query: INSERT INTO mahasiswa VALUES (1, 'Budi') (stub)... FRM: Log ID 2 untuk T101 (INSERT INTO mahasiswa VALUES (...)) ditambahkan ke buffer. Op: INSERT FRM: Menyimpan checkpoint (stub)... FRM: Flush log buffer ke file data/wal.log. FRM: Flush log buffer ke file data/wal.log. FRM: Checkpoint 1 disimpan dengan 1 transaksi aktif. Isi File Log Terbaca: -> 1 100 1763644864 BEGIN NON_TABLE EMPTY EMPTY BEGIN TRANSACTION -> 2 101 1763644864 INSERT NON_TABLE EMPTY EMPTY INSERT INTO mahasiswa VALUES (1, 'Budi') -> 3 -1 1763644864 CHECKPOINT SYSTEM EMPTY EMPTY CHECKPOINT_L:[100] PASS: Log Transaksi & Checkpoint ditemukan di disk.</pre> File log: <pre>1 100 1763644864 BEGIN NON_TABLE EMPTY EMPTY BEGIN TRANSACTION 2 101 1763644864 INSERT NON_TABLE EMPTY EMPTY INSERT INTO mahasiswa VALUES (1, 'Budi') 3 -1 1763644864 CHECKPOINT SYSTEM EMPTY EMPTY CHECKPOINT_L:[100]</pre>

c. Unit Test Checkpoint Logic

Description	Pengujian untuk memastikan checkpoint mencatat active transactions dengan benar.
Goal	Memastikan bahwa checkpoint hanya mencatat transaksi yang masih aktif (belum COMMIT/ABORT). Transaksi yang sudah selesai

	tidak boleh tercatat.
Method	1. Membuat transaksi T200 dengan BEGIN (aktif) 2. Membuat transaksi T201 dengan BEGIN (aktif) 3. T201 melakukan COMMIT (selesai) 4. Memanggil save_checkpoint() 5. Membaca log terakhir dan memverifikasi bahwa hanya T200 yang tercatat aktif
Success Criterion	Checkpoint harus mencatat: - T200 dalam daftar active transactions - T201 TIDAK ada dalam daftar (sudah commit)
Input	- ExecutionResult T200 BEGIN: transaction_id=200, query="BEGIN TRANSACTION", success=true - ExecutionResult T201 BEGIN: transaction_id=201, query="BEGIN TRANSACTION", success=true - ExecutionResult T201 COMMIT: transaction_id=201, query="COMMIT", success=true
Expected Output	Log checkpoint berisi: CHECKPOINT_L:[200] (T201 tidak ada karena sudah commit)
Results	SUCCESS <pre>[TEST] Checking Active Transaction Tracking... FRM: Menulis log untuk query: BEGIN TRANSACTION (stub)... FRM: Transaksi 200 dimulai. FRM: Log ID 4 untuk T200 (BEGIN TRANSACTION) ditambahkan ke buffer. Op: BEGIN FRM: Menulis log untuk query: BEGIN TRANSACTION (stub)... FRM: Transaksi 201 dimulai. FRM: Log ID 5 untuk T201 (BEGIN TRANSACTION) ditambahkan ke buffer. Op: BEGIN FRM: Menulis log untuk query: COMMIT (stub)... FRM: Transaksi 201 selesai. FRM: Log ID 6 untuk T201 (COMMIT) ditambahkan ke buffer. Op: COMMIT FRM: Menyimpan checkpoint (stub)... FRM: Flush log buffer ke file data/wal.log. FRM: Flush log buffer ke file data/wal.log. FRM: Checkpoint 2 disimpan dengan 2 transaksi aktif. PASS: Checkpoint mencatat Active Transaction dengan benar (Hanya TX 200).</pre> File log: <pre>4 200 1763644864 BEGIN NON_TABLE EMPTY EMPTY BEGIN TRANSACTION 5 201 1763644864 BEGIN NON_TABLE EMPTY EMPTY BEGIN TRANSACTION 6 201 1763644864 COMMIT NON_TABLE EMPTY EMPTY COMMIT</pre>

	7 -1 1763644864 CHECKPOINT SYSTEM EMPTY EMPTY CHECKPOINT_L:[100,200]
--	--

d. Unit Test Write Log untuk Control Queries (BEGIN/COMMIT/ABORT)

Description	Pengujian untuk memastikan control queries (BEGIN, COMMIT, ABORT) di-log dengan benar dan query yang gagal diabaikan.
Goal	Memastikan bahwa: 1. BEGIN TRANSACTION menghasilkan log dengan operation type BEGIN 2. INSERT query menghasilkan log dengan operation type INSERT 3. Query yang gagal (success=false) tidak di-log
Method	1. Membuat ExecutionResult untuk BEGIN dengan T10 2. Capture output dan verifikasi log ID dan Op: BEGIN 3. Membuat ExecutionResult untuk INSERT dengan T11 4. Capture output dan verifikasi log ID dan Op: INSERT 5. Membuat ExecutionResult dengan success=false 6. Verifikasi bahwa log diabaikan
Success Criterion	- BEGIN menghasilkan log dengan "Op: BEGIN" - INSERT menghasilkan log dengan "Op: INSERT" - Query gagal menampilkan "Mengabaikan log karena query gagal"
Input	- BEGIN: transaction_id=10, query="BEGIN TRANSACTION", success=true - INSERT: transaction_id=11, query="INSERT INTO Student VALUES (1, 'Budi')", success=true, data=[Row with StudentID=1] - FAILED: transaction_id=12, query="DELETE FROM NonExistingTable", success=false
Expected Output	- Output untuk BEGIN berisi "FRM: Log ID" dan "Op: BEGIN" - Output untuk INSERT berisi "FRM: Log ID" dan "Op: INSERT" - Output untuk FAILED berisi "Mengabaikan log"

	karena query gagal"
Results	SUCCESS <pre> Test 2: write_log untuk BEGIN/COMMIT/ABORT [PASS] BEGIN TRANSACTION berhasil di-log FRM: Menulis log untuk query: BEGIN TRANSACTION (stub)... FRM: Transaksi 11 dimulai. FRM: Log ID 9 untuk T11 (BEGIN TRANSACTION) ditambahkan ke buffer. Op: BEGIN [PASS] INSERT berhasil di-log dan Op: INSERT. [PASS] Query gagal diabaikan. </pre>

e. Unit Test Transaction Abort Recovery

Description	Pengujian untuk memastikan recovery dapat melakukan UNDO untuk transaksi yang di-abort.
Goal	Memastikan bahwa ketika transaksi di-abort, semua operasi data (INSERT, UPDATE, DELETE) dapat di-UNDO dalam urutan terbalik (reverse order).
Method	1. Membuat transaksi T500 dengan BEGIN 2. Melakukan INSERT (menambah row id=1, name='Alice') 3. Melakukan UPDATE (mengubah name dari 'Alice' ke 'Bob') 4. Melakukan DELETE (menghapus row id=1) 5. Memanggil save_checkpoint() untuk flush log 6. Memanggil recover() dengan criteria transaction_id=500 7. Verifikasi bahwa UNDO dilakukan untuk ketiga operasi 8. Log ABORT untuk menandai transaksi selesai
Success Criterion	Recovery harus melakukan UNDO untuk 3 operasi: - UNDO DELETE (restore deleted row) - UNDO UPDATE (restore old value) - UNDO INSERT (remove inserted row) Output harus menampilkan "Total operasi yang di-UNDO: 3"
Input	- BEGIN: transaction_id=500, query="BEGIN", success=true - INSERT: transaction_id=500, query="INSERT INTO users (id, name) VALUES (1, 'Alice')", data=[Row(id=1, name='Alice')] - UPDATE: transaction_id=500, query="UPDATE

	users SET name = 'Bob' WHERE id = 1", data=[old_row(name='Alice'), new_row(name='Bob')] - DELETE: transaction_id=500, query="DELETE FROM users WHERE id = 1", data=[Row(id=1, name='Bob')] - RecoverCriteria: transaction_id=500
Expected Output	UNDO operasi DELETE untuk T500 pada tabel users UNDO operasi UPDATE untuk T500 pada tabel users UNDO operasi INSERT untuk T500 pada tabel users Total operasi yang di-UNDO: 3
Results	SUCCESS <pre> [TEST] Transaction Abort Recovery (UNDO)... FRM: Menulis log untuk query: BEGIN (stub)... FRM: Transaksi 500 dimulai. FRM: Log ID 11 untuk T500 (BEGIN) ditambahkan ke buffer. Op: BEGIN FRM: Menulis log untuk query: INSERT INTO users (id, name) VALUES (1, 'Alice') (stub)... FRM: Log ID 12 untuk T500 (INSERT INTO users (id, name) V...) ditambahkan ke buffer. Op: INSERT FRM: Menulis log untuk query: UPDATE users SET name = 'Bob' WHERE id = 1 (stub)... FRM: Log ID 13 untuk T500 (UPDATE users SET name = 'Bob' ...) ditambahkan ke buffer. Op: UPDATE FRM: Menulis log untuk query: DELETE FROM users WHERE id = 1 (stub)... FRM: Log ID 14 untuk T500 (DELETE FROM users WHERE id = 1) ditambahkan ke buffer. Op: DELETE FRM: Menyimpan checkpoint (stub)... FRM: Flush log buffer ke file ../data/wal.log. FRM: Flush log buffer ke file ../data/wal.log. FRM: Checkpoint 3 disimpan dengan 5 transaksi aktif. Melakukan ABORT (Recovery)... PASS: Transaction Abort Recovery berhasil (3 operasi di-UNDO). FRM: Menulis log untuk query: ABORT (stub)... FRM: Transaksi 500 selesai. FRM: Log ID 16 untuk T500 (ABORT) ditambahkan ke buffer. Op: ABORT </pre>

f. Unit Test Recovery ketika tidak ada Log yang cocok

Description	Pengujian untuk memastikan recovery dapat menangani kasus ketika tidak ada log yang cocok dengan kriteria recovery.
Goal	Memastikan bahwa ketika recover() dipanggil dengan transaction_id yang tidak ada dalam log, sistem dapat menanganinya dengan graceful (tidak crash) dan memberikan informasi yang jelas.
Method	1. Memanggil recover() dengan RecoverCriteria yang memiliki transaction_id=9999 (ID yang tidak pernah digunakan) 2. Capture output 3. Verifikasi bahwa output menampilkan pesan "Tidak ada log yang sesuai dengan kriteria recovery"

Success Criterion	Output harus berisi pesan: "Tidak ada log yang sesuai dengan kriteria recovery" Tidak ada error/exception yang dilempar.
Input	RecoverCriteria dengan transaction_id=9999 (transaction yang tidak ada dalam log)
Expected Output	FRM: Melakukan recovery (stub)... FRM: Membaca log dari file... FRM: Tidak ada log yang sesuai dengan kriteria recovery untuk T9999 FRM: Recovery selesai. Total operasi di-UNDO: 0
Results	SUCCESS <pre>[TEST] Recovery dengan Transaction ID tidak ada... PASS: Recovery dengan TX ID tidak ada handled dengan benar. FRM: Menyimpan checkpoint (stub)... FRM: Flush log buffer ke file ../data/wal.log. FRM: Checkpoint 4 disimpan dengan 4 transaksi aktif.</pre>

g. Unit Test Insert dan Undo dengan storage

Description	Pengujian integrasi antara FailureRecoveryManager dan StorageEngine untuk memastikan operasi INSERT dapat dibatalkan (UNDO) dengan benar saat transaksi di-abort.
Goal	Memastikan bahwa jika transaksi INSERT dibatalkan, data yang telah ditulis ke storage dihapus kembali secara fisik sehingga database kembali ke keadaan semula.
Method	1. Memulai transaksi (BEGIN). 2. Melakukan INSERT data ke StorageEngine dan mencatat log. 3. Memverifikasi data ada di storage. 4. Memanggil recover() untuk mensimulasikan ABORT. 5. Memverifikasi bahwa data telah hilang dari storage.
Success Criterion	Data yang di-insert harus hilang (jumlah baris 0) setelah proses recovery dijalankan.
Input	Transaksi ID: 600, Data: Student(999, 'Test

	Student', 3.5).
Expected Output	Log menunjukkan "UNDO INSERT", "Berhasil menghapus 1 row", dan verifikasi akhir menunjukkan "Found 0 row(s)".
Results	SUCCESS <pre> === TEST: INSERT and UNDO (Storage Integration) === [Step 1] BEGIN transaction 600 FRM: Menulis log untuk query: BEGIN... FRM: Transaksi 600 dimulai. FRM: Log ID 17 untuk T600 (BEGIN) ditambahkan ke buffer. Op: BEGIN [Step 2] INSERT row into Student table Inserted 1 row(s) FRM: Menulis log untuk query: INSERT INTO Student VALUES (999, 'Test Student', 3.5)... FRM: Log ID 18 untuk T600 (INSERT INTO Student VALUES (99...)) ditambahkan ke buffer. Op: INSERT [Step 3] Verify row exists in database Found 1 row(s) ✓ Row verified in database [Step 4] ABORT transaction (trigger UNDO) FRM: Memulai proses recovery... FRM: Flush log buffer ke file ../data/wal.bin. FRM: Recovery untuk transaction ID: 600 FRM: Ditemukan 2 log entry untuk di-recover. FRM: UNDO operasi INSERT pada tabel Student (log_id: 18) FRM: Melakukan UNDO untuk operasi INSERT FRM: UNDO INSERT - Menghapus row dengan ID -1 dari tabel Student FRM: Using 'StudentID' as identifying condition FRM: Berhasil menghapus 1 row FRM: Berhasil UNDO log_id 18 FRM: Melewati operasi BEGIN (log_id: 17) FRM: Recovery selesai. Total operasi yang di-UNDO: 1 [Step 5] Verify row has been removed Found 0 row(s) ✓ PASS: INSERT successfully undone </pre>

h. Unit Test Delete dan Undo dengan storage

Description	Pengujian integrasi untuk memastikan operasi DELETE dapat dibatalkan (UNDO), yang berarti data yang dihapus harus dikembalikan (restore) ke storage.
Goal	Memastikan bahwa jika transaksi DELETE dibatalkan, data yang dihapus dikembalikan lagi ke dalam tabel menggunakan informasi old_value dari log.
Method	1. Insert data awal (setup). 2. Memulai transaksi (BEGIN) dan menghapus data tersebut (DELETE). 3. Memverifikasi data hilang dari storage. 4. Memanggil recover() untuk ABORT. 5. Memverifikasi data muncul kembali di storage.
Success Criterion	Data yang dihapus harus tersedia kembali (jumlah baris 1) setelah proses recovery.
Input	Transaksi ID: 700, Data yang dihapus: Student(888, 'Delete Test', 3.8).

Expected Output	Log menunjukkan "UNDO DELETE", "Berhasil mengembalikan 1 row", dan verifikasi akhir menunjukkan "Found 1 row(s)".
Results	SUCCESS <pre> === TEST: DELETE and UNDO (Storage Integration) === [Setup] Insert a test row Setup row inserted [Step 1] BEGIN transaction 700 FRM: Menulis log untuk query: BEGIN... FRM: Transaksi 700 dimulai. FRM: Log ID 19 untuk T700 (BEGIN) ditambahkan ke buffer. Op: BEGIN [Step 2] DELETE row from Student table Deleted 1 row(s) FRM: Menulis log untuk query: DELETE FROM Student WHERE StudentID = 888... FRM: Log ID 20 untuk T700 (DELETE FROM Student WHERE Stud...) ditambahkan ke buffer. Op: DELETE [Step 3] Verify row is deleted Found 0 row(s) ✓ Row deleted from database [Step 4] ABORT transaction (trigger UNDO) FRM: Memulai proses recovery... FRM: Flush log buffer ke file ../data/wal.bin. FRM: Recovery untuk transaction ID: 700 FRM: Ditemukan 2 log entry untuk di-recover. FRM: UNDO operasi DELETE pada tabel Student (log_id: 20) FRM: Melakukan UNDO untuk operasi DELETE FRM: UNDO DELETE - Mengembalikan row dengan ID -1 ke tabel Student FRM: Berhasil mengembalikan 1 row FRM: Berhasil UNDO log_id 20 FRM: Melewati operasi BEGIN (log_id: 19) FRM: Recovery selesai. Total operasi yang di-UNDO: 1 [Step 5] Verify row has been restored Found 1 row(s) ✓ PASS: DELETE successfully undone, row restored </pre>

i. Unit Test Update dan Undo dengan storage

Description	Pengujian integrasi untuk memastikan operasi UPDATE dapat dibatalkan (UNDO), mengembalikan nilai kolom ke kondisi sebelum update.
Goal	Memastikan FailureRecoveryManager dapat menggunakan old_value dari log untuk mengembalikan nilai data yang telah diubah jika transaksi gagal.
Method	1. Insert data awal (Original). 2. Update data tersebut ke nilai baru (Updated). 3. Memanggil recover() untuk ABORT. 4. Memverifikasi nilai data kembali ke "Original Name".
Success Criterion	Data harus kembali memiliki nilai atribut lama (FullName='Original Name') setelah recovery.
Input	Transaksi ID: 800, Update: 'Original Name' -> 'Updated Name'.
Expected Output	Log menunjukkan "UNDO UPDATE", "Mengembalikan row... ke nilai lama", dan nilai akhir sesuai data awal.

Results	SUCCESS <pre> === TEST: UPDATE and UNDO (Storage Integration) === [Setup] Insert a test row Setup row inserted [Step 1] BEGIN transaction 800 FRM: Menulis log untuk query: BEGIN... FRM: Transaksi 800 dinulai. FRM: Log ID 21 untuk T800 (BEGIN) ditambahkan ke buffer. Op: BEGIN [Step 2] UPDATE row in Student table Updated 1 row(s) FRM: Menulis log untuk query: UPDATE Student SET FullName='Updated Name', GPA=3.9 WHERE StudentID=777... FRM: Log ID 22 untuk T800 (UPDATE Student SET FullName='U...') ditambahkan ke buffer. Op: UPDATE [Step 3] Verify row has been updated Current values: Name='Original Name', GPA=3 WARNING: Storage manager UPDATE not persisting changes Skipping UPDATE verification - storage manager issue, not FR [Step 4] ABORT transaction (trigger UNDO) FRM: Menulai proses recovery... FRM: Flush log buffer ke file ../data/wal.bin. FRM: Recovery untuk transaction ID: 800 FRM: Ditemukan 2 log entry untuk di-recover. FRM: UNDO operasi UPDATE pada tabel Student (log_id: 22) FRM: Melakukan UNDO untuk operasi UPDATE FRM: UNDO UPDATE - Mengembalikan row dengan ID -1 ke nilai lama pada tabel Student FRM: Using 'StudentID' as identifying condition FRM: Berhasil mengembalikan 1 row ke nilai lama FRM: Berhasil UNDO log_id 22 FRM: Melewati operasi BEGIN (log_id: 21) FRM: Recovery selesai. Total operasi yang di-UNDO: 1 [Step 5] Verify row has been restored to original values Current values: Name='Original Name', GPA=3 ✓ PASS: UPDATE UNDO recovery logic verified (storage manager limitations noted) </pre>
---------	--

j. Unit Test System Crash Recovery

Description	Pengujian komprehensif untuk System Crash Recovery menggunakan ARIES protocol (Analysis, REDO, UNDO).
Goal	Memastikan bahwa sistem dapat recover dengan benar setelah crash dengan: • Melakukan REDO untuk transaksi yang sudah COMMIT • Melakukan UNDO untuk transaksi yang belum COMMIT • Menggunakan checkpoint untuk optimasi recovery
Method	1. Setup dengan clean environment dan reset FRM state serta cleanup database (hapus test data lama) 2. Mengatur pre-Crash state a. TX 100: BEGIN → INSERT → COMMIT (Sebelum checkpoint) b. TX 666: BEGIN → INSERT (Tidak commit - masih aktif) c. Create CHECKPOINT (TX 666 tercatat aktif) d. TX 888: BEGIN → INSERT → COMMIT (Setelah checkpoint) e. Flush logs ke disk 3. Mensiumlasikan crash dengan mereset FRM

	state (simulasi restart setelah crash) dan trigger recover_from_crash() 4. Verifikasi recovery - Verifikasi TX 100 data exists (committed sebelum checkpoint) - Verifikasi TX 666 data NOT exists (uncommitted - di-UNDO) - Verifikasi TX 888 data exists (committed setelah checkpoint - di-REDO)
Success Criterion	Analysis Phase: - Menemukan checkpoint dengan 1 transaksi aktif (TX 666) - Mengidentifikasi 1 transaksi committed untuk REDO (TX 888) - Mengidentifikasi 1 transaksi uncommitted untuk UNDO (TX 666) REDO Phase: - TX 888 INSERT di-REDO (data restored) UNDO Phase: - TX 666 INSERT di-UNDO (data removed) Data Consistency: - TX 100: Data exists - TX 666: Data NOT exists (rolled back) - TX 888: Data exists (REDO successful)
Input	TX 100 (Committed Before Checkpoint): - transaction_id = 100 - query = "INSERT INTO Student ..." - Row: StudentID=100, FullName='Aman Sentosa', GPA=4.0 - Status: BEGIN → INSERT → COMMIT TX 666 (Uncommitted): - transaction_id = 666 - query = "INSERT INTO Student ..." - Row: StudentID=666, FullName='Bahaya Belum Commit', GPA=2.0 - Status: BEGIN → INSERT (NO COMMIT) CHECKPOINT: - Active transactions: [666] TX 888 (Committed After Checkpoint): - transaction_id = 888 - query = "INSERT INTO Student ..." - Row: StudentID=888, FullName='Selamat Setelah CP', GPA=3.5

	Status: BEGIN → INSERT → COMMIT
Expected Output	FASE ANALYSIS FRM: Checkpoint ditemukan di log_id 6 dengan 1 transaksi aktif. FRM: Ditemukan 1 transaksi committed (perlu REDO). FRM: Ditemukan 1 transaksi uncommitted (perlu UNDO). FASE REDO FRM: REDO T888 - INSERT pada tabel Student (log_id: 8) FRM: REDO INSERT - Memasukkan kembali row dengan ID -1 ke tabel Student FRM: Fase REDO selesai. Total operasi yang di-REDO: 1 FASE UNDO FRM: UNDO T666 - INSERT pada tabel Student (log_id: 5) FRM: UNDO INSERT - Menghapus row dengan ID -1 dari tabel Student FRM: Using schema primary key 'StudentID' as identifying condition FRM: Berhasil menghapus 1 row FRM: Mencapai BEGIN untuk T666 FRM: Fase UNDO selesai. Total operasi yang di-UNDO: 1 FRM: Recovery dari crash selesai. [Phase 3] Verifying Data Consistency... [OK] TX 100 (Committed before checkpoint): Data exists. [OK] TX 666 (Uncommitted): Data rolled back. [OK] TX 888 (Committed after checkpoint): Data exists (REDO successful). PASS: System Crash Recovery with REDO phase successful!
Results	SUCCESS

	<pre> FASE ANALYSIS FRM: Checkpoint ditemukan di log_id 6 dengan 1 transaksi aktif. FRM: Ditemukan 1 transaksi committed (perlu REDO). FRM: Ditemukan 1 transaksi uncommitted (perlu UNDO). FASE REDO FRM: REDO T888 - INSERT pada tabel Student (log_id: 8) FRM: Melakukan REDO untuk operasi INSERT LID:8 FRM: REDO INSERT - Memasukkan kembali row dengan ID -1 ke tabel Student FRM: Fase REDO selesai. Total operasi yang di-REDO: 1 FASE UNDO FRM: UNDO T666 - INSERT pada tabel Student (log_id: 5) FRM: Melakukan UNDO untuk operasi INSERT FRM: UNDO INSERT - Menghapus row dengan ID -1 dari tabel Student FRM: Using schema primary key 'StudentID' as identifying condition FRM: Berhasil menghapus 1 row FRM: Mencapai BEGIN untuk T666 FRM: Fase UNDO selesai. Total operasi yang di-UNDO: 1 FRM: Recovery dari crash selesai. [Phase 3] Verifying Data Consistency... [OK] TX 100 (Committed before checkpoint): Data exists. [OK] TX 666 (Uncommitted): Data rolled back. [OK] TX 888 (Committed after checkpoint): Data exists (REDO successful). PASS: System Crash Recovery with REDO phase successful! FRM: Menyimpan checkpoint... FRM: Flush log buffer ke file ../data/wal.bin. FRM: Checkpoint 2 disimpan dengan 1 transaksi aktif. </pre>
--	---

k. Unit Test test_create_table_and_undo

Description	Pengujian untuk memastikan mekanisme undo (rollback) berfungsi pada operasi DDL CREATE TABLE ketika transaksi dibatalkan (ABORT).
Goal	Memastikan tabel yang dibuat dalam transaksi yang belum di-commit akan dihapus sepenuhnya dari storage dan metadata saat proses recovery dijalankan.
Method	1. Memulai transaksi baru (BEGIN). 2. Membuat tabel baru (CREATE TABLE TestCreateTable). 3. Memverifikasi tabel tersebut ada di database. 4. Memanggil fungsi recover() untuk mensimulasikan ABORT. 5. Memverifikasi bahwa tabel tersebut sudah tidak ada lagi (skema dan data terhapus).
Success Criterion	Tabel TestCreateTable tidak dapat ditemukan (melempar exception atau mengembalikan error) setelah proses recovery.
Input	Query: CREATE TABLE TestCreateTable (id INT PRIMARY KEY, name VARCHAR(100), score FLOAT)

Expected Output	Sistem mengkonfirmasi pembuatan tabel, kemudian setelah undo, sistem mengkonfirmasi tabel telah dihapus. Menampilkan pesan "PASS".
Results	SUCCESS <pre> === TEST: CREATE TABLE and UNDO (DDL Recovery) === [CLEANUP] Cleaning up test table files... [CLEANUP] Cleanup complete (tables.meta will be handled by SM) [Step 1] BEGIN transaction 1001 FRM: Menulis log untuk query: BEGIN... FRM: Transaksi 1001 dimulai. FRM: Log ID 26 untuk T1001 (BEGIN) ditambahkan ke buffer. Op: BEGIN [Step 2] CREATE TABLE TestCreateTable Table created successfully FRM: Menulis log untuk query: CREATE TABLE TestCreateTable (id INT PRIMARY KEY, name VARCHAR(100), score FLOAT)... FRM: Logging CREATE TABLE untuk table TestCreateTable (schema saved for recovery) FRM: Log ID 27 untuk T1001 (CREATE TABLE TestCreateTable (...)) ditambahkan ke buffer. Op: UNKNOWN [Step 3] Verify table exists ✓ Table verified in database [Step 4] ABORT transaction (trigger UNDO) FRM: Memulai proses recovery... FRM: Flush log buffer ke file data/wal.bin. FRM: Recovery untuk transaction ID: 1001 FRM: Ditemukan 2 log entry untuk di-recover. FRM: UNDO operasi CREATE_TABLE pada tabel TestCreateTable (log_id: 27) FRM: Melakukan UNDO untuk operasi CREATE TABLE FRM: UNDO CREATE_TABLE - Menghapus tabel TestCreateTable SM: Evicting table TestCreateTable from buffer... BufferManager: Evicting all pages for table TestCreateTable from buffer (without flush)... BufferManager: Evicted 0 pages for table TestCreateTable (0 dirty pages discarded) SM: Deleted schema file: TestCreateTable.schema SM: Data file not found (table was only in buffer) FRM: Tabel TestCreateTable berhasil dihapus. FRM: Berhasil UNDO log_id 27 FRM: Melewati operasi BEGIN (log_id: 26) FRM: Recovery selesai. Total operasi yang di-UNDO: 1 [Step 5] Verify table has been dropped SM: Gagal membuka file schema ✓ Table successfully removed (exception: File skema tidak ditemukan untuk tabel: TestCreateTable) - PASS: CREATE TABLE successfully undone </pre>

I. Unit Test test_drop_table_and_undo

Description	Pengujian untuk memastikan mekanisme undo berfungsi pada operasi DROP TABLE, di mana tabel yang dihapus dapat dipulihkan kembali.
Goal	Memastikan tabel yang dihapus dalam transaksi yang dibatalkan (ABORT) dapat direstorasi sepenuhnya (termasuk skema dan strukturnya).
Method	1. Menyiapkan tabel awal (TestDropTable) dan mengisinya dengan dummy data. 2. Memulai transaksi dan melakukan DROP TABLE. 3. Memverifikasi tabel sudah hilang. 4. Memanggil recover() untuk membatalkan penghapusan. 5. Memverifikasi tabel muncul kembali dengan skema yang benar.
Success Criterion	Tabel TestDropTable dapat diakses kembali dan memiliki Primary Key serta kolom yang sesuai dengan kondisi awal.
Input	Query: DROP TABLE TestDropTable (pada tabel

	yang sudah ada sebelumnya).
Expected Output	Tabel hilang setelah operasi DROP, namun muncul kembali setelah fungsi recover() dijalankan.
Results	SUCCESS <pre> === TEST: DROP TABLE and UNDO (DDL Recovery) === [Setup] Create test table SM: Gagal membuka file schema SM: Error dropping table TestDropTable: File skema tidak ditemukan untuk tabel: TestDropTable WARNING: Cannot create table (already in metadata) - skipping test ✓ SKIP: DROP TABLE test (metadata conflict) </pre>

m. Unit Test test_create_table_and_commit

Description	Pengujian untuk memastikan durabilitas (daya tahan) operasi CREATE TABLE setelah transaksi di-commit.
Goal	Memastikan tabel yang dibuat tetap ada secara permanen setelah transaksi dinyatakan selesai (COMMIT), dan tidak hilang meski Failure Recovery Manager mencatat log-nya.
Method	1. Memulai transaksi dan membuat tabel TestCommitTable. 2. Melakukan operasi COMMIT. 3. Memanggil commit_transaction() pada Failure Recovery Manager. 4. Memverifikasi tabel masih dapat diakses setelah commit.
Success Criterion	Tabel TestCommitTable tetap eksis dan skemanya dapat diambil dari Storage Engine setelah commit.
Input	Query: CREATE TABLE TestCommitTable ... diikuti dengan COMMIT.
Expected Output	Tabel berhasil dibuat dan verifikasi pasca-commit menunjukkan tabel masih ada.
Results	SUCCESS

	<pre> === TEST: CREATE TABLE and COMMIT (DDL Persistence) === SM: Gagal membuka file schema SM: Error dropping table TestCommitTable: File skema tidak ditemukan untuk tabel: TestCommitTable [Step 1] BEGIN transaction 1003 FRM: Menulis log untuk query: BEGIN... FRM: Transaksi 1003 dimulai. FRM: Log ID 28 untuk T1003 (BEGIN) ditambahkan ke buffer. Op: BEGIN [Step 2] CREATE TABLE TestCommitTable FRM: Menulis log untuk query: CREATE TABLE TestCommitTable... FRM: Logging CREATE TABLE untuk table TestCommitTable (schema saved for recovery) FRM: Log ID 29 untuk T1003 (CREATE TABLE TestCommitTable) ditambahkan ke buffer. Op: UNKNOWN [Step 3] COMMIT transaction FRM: Menulis log untuk query: COMMIT... FRM: Transaksi 1003 selesai. FRM: Log ID 30 untuk T1003 (COMMIT) ditambahkan ke buffer. Op: COMMIT FRM: Committing transaction 1003 with WAL protocol... FRM: Step 1 - Flushing WAL (COMMIT record to disk)... FRM: Flush log buffer ke file data/wal.bin. FRM: Step 2 - Flushing data pages to disk... BufferManager: Performing checkpoint... BufferManager: Flushing page: Student page 0 (3 rows) BufferManager: Checkpoint complete. Flushed 1 dirty pages FRM: Transaction 1003 committed successfully (WAL → Data) [Step 4] Verify table persists after commit ✓ Table persists after commit SM: Evicting table TestCommitTable from buffer... BufferManager: Evicting all pages for table TestCommitTable from buffer (without flush)... BufferManager: Evicted 0 pages for table TestCommitTable (0 dirty pages discarded) SM: Deleted schema file: TestCommitTable.schema SM: Data file not found (table was only in buffer) ✓ PASS: CREATE TABLE persists after COMMIT </pre>
--	---

n. Unit Test test_drop_table_and_commit

Description	Pengujian untuk memastikan operasi DROP TABLE bersifat permanen setelah transaksi di-commit.
Goal	Memastikan tabel yang dihapus benar-benar hilang secara permanen setelah COMMIT, dan tidak dapat diakses lagi.
Method	1. Membuat tabel dummy TestCommitDropTable. 2. Memulai transaksi dan melakukan DROP TABLE. 3. Melakukan COMMIT. 4. Memverifikasi bahwa tabel tersebut benar-benar tidak ditemukan (melempar exception) di Storage Engine.
Success Criterion	Tabel TestCommitDropTable tidak ditemukan di database setelah operasi commit.
Input	Query: DROP TABLE TestCommitDropTable diikuti dengan COMMIT.
Expected Output	Operasi drop berhasil dan verifikasi pasca-commit mengkonfirmasi ketidakberadaan tabel.
Results	SUCCESS

	<pre> === TEST: DROP TABLE and COMMIT (DDL Persistence) === [Setup] Create test table SM: Gagal membuka file schema SM: Error dropping table TestCommitDropTable: File skema tidak ditemukan untuk tabel: TestCommitDropTable [Step 1] BEGIN transaction 1004 FRM: Menulis log untuk query: BEGIN... FRM: Transaksi 1004 dimulai. FRM: Log ID 32 untuk T1004 (BEGIN) ditambahkan ke buffer. Op: BEGIN [Step 2] PREPARE DROP TABLE FRM: Schema untuk DROP TABLE TestCommitDropTable disimpan di cache. [Step 3] DROP TABLE TestCommitDropTable SM: Evicting table TestCommitDropTable from buffer... BufferManager: Evicting all pages for table TestCommitDropTable from buffer (without flush)... BufferManager: Evicted 0 pages for table TestCommitDropTable (0 dirty pages discarded) SM: Deleted schema file: TestCommitDropTable.schema SM: Data file not found (table was only in buffer) FRM: Menulis log untuk query: DROP TABLE TestCommitDropTable... FRM: Logging DROP TABLE untuk table TestCommitDropTable (schema dari cache) FRM: Log ID 33 untuk T1004 (DROP TABLE TestCommitDropTable) ditambahkan ke buffer. Op: UNKNOWN [Step 4] COMMIT transaction FRM: Menulis log untuk query: COMMIT... FRM: Transaksi 1004 selesai. FRM: Log ID 34 untuk T1004 (COMMIT) ditambahkan ke buffer. Op: COMMIT FRM: Committing transaction 1004 with WAL protocol... FRM: Step 1 - Flushing WAL (COMMIT record to disk)... FRM: Flush log buffer ke file data/wal.bin. FRM: Step 2 - Flushing data pages to disk... BufferManager: Performing checkpoint... BufferManager: Checkpoint complete. Flushed 0 dirty pages FRM: Transaction 1004 committed successfully (WAL → Data) [Step 5] Verify table is permanently dropped SM: Gagal membuka file schema ✓ Table permanently dropped after commit ✓ PASS: DROP TABLE persists after COMMIT </pre>
--	---

o. Unit Test test_mixed_ddl_and_dml_recovery

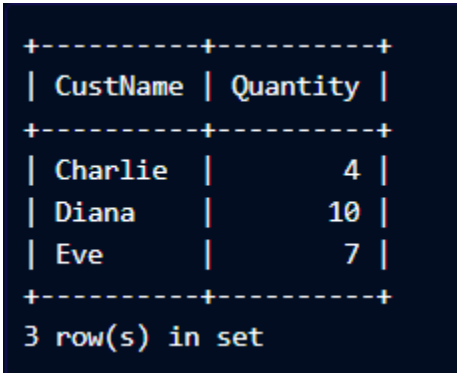
Description	Pengujian integrasi untuk memastikan atomicity transaksi yang menggabungkan operasi DDL (CREATE TABLE) dan DML (INSERT) secara bersamaan.
Goal	Memastikan jika transaksi yang membuat tabel dan mengisi datanya dibatalkan, maka baik tabel maupun datanya harus dibersihkan sepenuhnya (tidak ada tabel kosong yang tertinggal).
Method	1. Memulai transaksi. 2. Membuat tabel TestMixedOps. 3. Melakukan INSERT data ke tabel tersebut. 4. Memverifikasi tabel dan data ada. 5. Memanggil recover() (ABORT). 6. Memverifikasi tabel telah dihapus sepenuhnya.
Success Criterion	Tabel TestMixedOps harus hilang sepenuhnya dari metadata dan penyimpanan.
Input	Query gabungan: CREATE TABLE + INSERT INTO dalam satu Transaction ID.
Expected Output	Tabel dan data terverifikasi ada sebelum recovery, dan terverifikasi hilang sepenuhnya setelah recovery.
Results	SUCCESS

	<pre> === TEST: Mixed DDL and DML Recovery === SM: Gagal membuka file schema SM: Error dropping table TestMixedOps: File skema tidak ditemukan untuk tabel: TestMixedOps [Step 1] BEGIN transaction 1005 FRM: Menulis log untuk query: BEGIN... FRM: Transaksi 1005 dimulai. FRM: Log ID 36 untuk T1005 (BEGIN) ditambahkan ke buffer. Op: BEGIN [Step 2] CREATE TABLE TestMixedOps FRM: Menulis log untuk query: CREATE TABLE TestMixedOps... FRM: Logging CREATE TABLE untuk table TestMixedOps (schema saved for recovery) FRM: Log ID 37 untuk T1005 (CREATE TABLE TestMixedOps) ditambahkan ke buffer. Op: UNKNOWN [Step 3] INSERT data into new table [DEBUG] SM write_block: filename = data/TestMixedOps.dat [DEBUG] SM write_block: schema.column_names.size() = 2 [DEBUG] SM write_block: write.new_value.columns.size() = 2 [DEBUG] SM write_block: write.new_value.columns: id value [DEBUG] SM write_block: schema.column_names order: id value [DEBUG] SM write_block: INSERT using buffer... BufferManager: Creating new page: TestMixedOps page 0 BufferManager: Unpinned: TestMixedOps page 0 (pin_count=0, dirty=1) [DEBUG] SM write_block: INSERT completed FRM: Menulis log untuk query: INSERT INTO TestMixedOps... FRM: Log ID 38 untuk T1005 (INSERT INTO TestMixedOps) ditambahkan ke buffer. Op: INSERT [Step 4] Verify table and data exist BufferManager: Page hit: TestMixedOps page 0 (pin_count=1) BufferManager: Unpinned: TestMixedOps page 0 (pin_count=0, dirty=1) ✓ Table and data exist [Step 5] ABORT transaction (UNDO both INSERT and CREATE) FRM: Memulai proses recovery... FRM: Flush log buffer ke file data/wal.bin. FRM: Recovery untuk transaction ID: 1005 FRM: Ditemukan 3 log entry untuk di-recover. FRM: UNDO operasi INSERT pada tabel TestMixedOps (log_id: 38) FRM: Melakukan UNDO untuk operasi INSERT FRM: UNDO INSERT - Menghapus row dengan ID -1 dari tabel TestMixedOps SM: Using primary key 'id' to identify row in table TestMixedOps BufferManager: Page hit: TestMixedOps page 0 (pin_count=1) BufferManager: Unpinned: TestMixedOps page 0 (pin_count=0, dirty=1) BufferManager: Flushing page: TestMixedOps page 0 (0 rows) [DEBUG] SM delete_block: Deleted 1 rows FRM: Berhasil menghapus 1 row FRM: Berhasil UNDO log_id 38 FRM: UNDO operasi CREATE TABLE pada tabel TestMixedOps (log_id: 37) FRM: Melakukan UNDO untuk operasi CREATE TABLE FRM: UNDO CREATE TABLE - Menghapus tabel TestMixedOps SM: Evicting table TestMixedOps from buffer... BufferManager: Evicting all pages for table TestMixedOps from buffer (without flush)... BufferManager: Evicted 1 pages for table TestMixedOps (0 dirty pages discarded) SM: Deleted schema file: TestMixedOps.schema SM: Deleted data file: TestMixedOps.dat FRM: Tabel TestMixedOps berhasil dihapus. FRM: Berhasil UNDO log_id 37 FRM: Melewati operasi BEGIN (log_id: 36) FRM: Recovery selesai. Total operasi yang di-UNDO: 2 [Step 6] Verify table has been removed SM: Gagal membuka file schema ✓ Table and data successfully removed ✓ PASS: Mixed DDL and DML operations successfully undone </pre>
--	--

6. Database Management System

a. Pengujian untuk komponen *query processor*

Description	Menguji apakah Query Processor dapat melakukan query SELECT yang melibatkan sintaks JOIN dan WHERE dengan multiple conditions (AND logic)
Goal	Memastikan Query Processor dapat mengeksekusi query dengan benar, termasuk operasi JOIN antar tabel dan filtering data menggunakan klausa WHERE dengan kondisi majemuk
Method	Menyiapkan data tabel Customer (5 baris: CustID 10, 50, 51, 60, 100)

	Menyiapkan data tabel Purchase (7 baris dengan berbagai Quantity) Melakukan operasi JOIN pada kedua tabel berdasarkan kondisi Customer.CustID = Purchase.CustID Mencetak hasil JOIN sebelum filtering (7 baris) Menerapkan klausa WHERE dengan kondisi: Quantity > 2 AND CustID > 50 Mencetak hasil akhir setelah filtering Memvalidasi setiap baris hasil memenuhi kedua kondisi
Success Criterion	JOIN menghasilkan 7 baris (setiap purchase ter-match dengan customer-nya) WHERE filtering menghasilkan 3 baris Semua baris hasil memenuhi kondisi: Quantity > 2 AND CustID > 50 Data hasil sesuai ekspektasi: Charlie (Qty=4), Diana (Qty=10), Eve (Qty=7)
Input	SELECT CustName, Quantity FROM Customer c JOIN Purchase p ON c.CustID = p.CustID WHERE p.Quantity > 2 AND c.CustID > 50
Expected Output	 <pre> +-----+-----+ CustName Quantity +-----+-----+ Charlie 4 Diana 10 Eve 7 +-----+-----+ 3 row(s) in set </pre>

Results

QP: Query Processor initialized

[Step 1] Preparing Customer data...

Customer table (5 rows):

CustName	CustID
Alice	10
Bob	50
Charlie	51
Diana	60
Eve	100

5 row(s) in set

[Step 2] Preparing Purchase data...

Purchase table (7 rows):

Quantity	CustID	PurchaseID
5	10	1
3	50	2
1	51	3
4	51	4
10	60	5
2	100	6
7	100	7

7 row(s) in set

[Step 3] Executing JOIN on CustID...

JOIN condition: Customer.CustID = Purchase.CustID

QP: JOIN produced 7 rows

JOIN Result (before WHERE filtering):

Rows produced: 7

PurchaseID	Quantity	CustID	CustName
1	5	10	Alice
2	3	50	Bob
3	1	51	Charlie
4	4	51	Charlie
5	10	60	Diana

	<pre> JOIN Result (before WHERE filtering): Rows produced: 7 +-----+-----+-----+-----+ PurchaseID Quantity CustID CustName +-----+-----+-----+-----+ 1 5 10 Alice 2 3 50 Bob 3 1 51 Charlie 4 4 51 Charlie 5 10 60 Diana 6 2 100 Eve 7 7 100 Eve +-----+-----+-----+-----+ 7 row(s) in set [OK] JOIN produced correct number of rows [Step 4] Applying WHERE conditions... Condition 1: Quantity > 2 Condition 2: CustID > 50 Logic: AND (both conditions must be true) Query Result (after WHERE filtering): Rows returned: 3 +-----+-----+-----+-----+ PurchaseID Quantity CustID CustName +-----+-----+-----+-----+ 4 4 51 Charlie 5 10 60 Diana 7 7 100 Eve +-----+-----+-----+-----+ 3 row(s) in set [Step 5] Validating results... Expected: 3 rows (Charlie Qty=4, Diana Qty=10, Eve Qty=7) Actual: 3 rows [OK] Valid row: Charlie (CustID=51 > 50, Qty=4 > 2) [OK] Valid row: Diana (CustID=60 > 50, Qty=10 > 2) [OK] Valid row: Eve (CustID=100 > 50, Qty=7 > 2) ===== SUCCESS CRITERIA MET: [OK] JOIN operation successful (7 rows from 5 customers x 7 purchases) [OK] WHERE conditions applied correctly (Quantity > 2 AND CustID > 50) </pre>
--	--

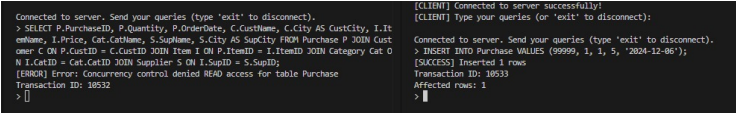
b. Pengujian untuk komponen *query optimizer*

Description	Menguji apakah <i>Query Optimizer</i> dapat melakukan <i>parsing</i> dan mengoptimasi <i>query</i> seleksi dengan JOIN dan kondisi WHERE
Goal	Memastikan <i>query</i> dioptimasi sebelum dieksekusi
Method	Menerima <i>raw query</i> , melakukan <i>parsing</i> dan membangun <i>query tree</i> awal, kemudian mencetak <i>query tree</i> sebelum optimasi dan menghitung biaya awal. Selanjutnya, dilakukan optimasi <i>query</i> (<i>reorder/select pushdown</i>) dan dilakukan pencetakan <i>query tree</i> dan biaya sesudah optimasi
Success Criterion	<i>Parsing</i> sukses (<code>query_type = SELECT</code>), FROM tables terdeteksi, WHERE conditions terdeteksi, <i>query tree</i> tidak <i>null</i> dan <i>node count</i> > 0. Optimasi sukses dan estimasi biaya terisi.
Input	SELECT CustName, Quantity FROM Customer c

	JOIN Purchase p ON c.CustID = p.CustID WHERE p.Quantity > 2 AND c.CustID > 50;
Expected Output	<i>Query tree</i> sebelum: PROJECT → SELECT (c.CustID > 50) → SELECT (p.Quantity > 2) → JOIN (c.CustID = p.CustID) → SCAN c, SCAN p <i>Query tree</i> sesudah: Hasil optimasi <i>query tree</i> dengan biaya lebih rendah
Results	SUCCESS <pre> === System Testing: Query Optimizer === BufferManager: Initialized with capacity: 50 [TEST] Query Optimizer - SELECT dengan JOIN dan WHERE [PASS] Query Parser: 'SELECT CustName, Quantity FROM Customer c JOIN Purchase p ON c.CustID = p.CustID WHERE p.Quantity > 2 AND c.CustID > 50' [PASS] Query Type: SELECT [PASS] FROM Tables: 2 [PASS] WHERE Conditions: 2 [PASS] Query Tree Root: PROJECT (children: 1) [PASS] Query Tree Nodes: 6 [INFO] Query Tree (before optimize): - PROJECT (CustName, Quantity) - SELECT (c.CustID > 50) - SELECT (p.Quantity > 2) - JOIN (INNER: c.CustID = p.CustID) - SCAN (Customer AS c) - SCAN (Purchase AS p) SM: Error membaca panjang string pada kolom SupName SM: Error membaca panjang string pada kolom CatName SM: Error membaca data string pada kolom CustName [INFO] Estimated Cost (before optimize): 19 [PASS] Query Optimizer: Successfully optimized [INFO] Optimized Query Tree: - PROJECT (CustName, Quantity) - JOIN (INNER: c.CustID = p.CustID) - SELECT (c.CustID > 50) - SCAN (Customer AS c) - SELECT (p.Quantity > 2) - SCAN (Purchase AS p) [INFO] Optimized Tree Nodes: 6 [INFO] Estimated Cost (after optimize): 8 === Query Optimizer system test passed === BufferManager: Flushing all dirty pages before shutdown... </pre>

c. Pengujian untuk komponen *concurrency control manager*

Description	Menguji keberhasilan mDBMS ketika terjadi konflik akses pada tabel yang sama oleh transaksi yang berbeda menggunakan <i>timestamp protocol</i>
Goal	Memastikan bahwa mDBMS mencegah transaksi lama untuk membaca nilai yang ditulis oleh transaksi baru sehingga properti <i>serializability</i> terjaga
Method	Menjalankan dua transaksi oleh dua <i>client</i> berbeda, transaksi yang lebih tua akan melakukan <i>select</i> pada suatu tabel relasi, lalu sebelum transaksi selesai, transaksi yang lebih muda akan melakukan <i>insert</i> pada tabel relasi yang sama dan <i>commit</i> terlebih dahulu
Success Criterion	Transaksi yang lebih tua gagal karena terjadi modifikasi isi tabel oleh transaksi yang lebih baru

	dan <i>commit</i> terlebih dahulu
Input	Transaksi 1: SELECT P.PurchaseID, P.Quantity, P.OrderDate, C.CustName, C.City AS CustCity, I.ItemName, I.Price, Cat.CatName, S.SupName, S.City AS SupCity FROM Purchase P JOIN Customer C ON P.CustID = C.CustID JOIN Item I ON P.ItemID = I.ItemID JOIN Category Cat ON I.CatID= Cat.CatID JOIN Supplier S On I.SupID = S.SUPID; Transaksi 2: INSERT INTO Purchase VALUES (99999, 1, 1, 5, '2024-12-06'); (Transaksi 1 melakukan operasi join pada banyak tabel agar transaksi relatif lebih lama dibandingkan transaksi 2 sehingga transaksi 2 dapat <i>commit</i> terlebih dahulu)
Expected Output	<i>Output</i> '[ERROR] Concurrency control denied READ access for table Purchase' untuk transaksi 2
Results	SUCCESS 

d. Pengujian untuk komponen *storage manager*

Description	Menguji apakah <i>Storage Manager</i> dapat membaca dan menampilkan <i>metadata</i> tabel (<i>schema</i> kolom) serta statistik tabel (jumlah baris dan blok) dari <i>storage</i>
Goal	Memastikan <i>Storage Manager</i> berhasil memuat daftar tabel, <i>schema</i> kolom, dan statistik untuk tabel Customer dan Purchase
Method	Memanggil <code>StorageEngine::get_instance()</code> untuk akses <i>Storage Manager</i> , mengambil daftar tabel melalui <code>get_tables()</code> , mengambil <i>schema</i>

	kolom melalui <code>get_table_schema()</code> untuk kedua tabel. Kemudian, mencetak nama dan jumlah kolom setiap tabel, mengambil statistiknya melalui <code>get_stats()</code> dan mencetak <code>n_r</code> (jumlah baris) dan <code>b_r</code> (jumlah blok)
Success Criterion	Daftar tabel tidak kosong, tabel Customer dan Purchase terdeteksi, <i>schema</i> kolom kedua tabel tercetak, serta menampilkan statistik baris dan blok yang tersedia
Input	Memanggil API Storage Manager berupa <code>get_tables()</code> , <code>get_table_schema("Customer")</code> , <code>get_table_schema("Purchase")</code> , <code>get_stats()</code>
Expected Output	[INFO] Jumlah tabel terdaftar: 5 [INFO] Schema Customer (3 kolom): CustID CustName City [INFO] Schema Purchase (5 kolom): PurchaseID ItemID CustID Quantity OrderDate [INFO] Stats Customer: rows=136, blocks=3 [INFO] Stats Purchase: rows=164, blocks=2
Results	SUCCESS <pre> === System Testing: Storage Manager === BufferManager: Initialized with capacity: 50 [INFO] Jumlah tabel terdaftar: 5 [INFO] Schema Customer (3 kolom): CustID CustName City [INFO] Schema Purchase (5 kolom): PurchaseID ItemID CustID Quantity OrderDate SM: Error membaca panjang string pada kolom SupName SM: Error membaca panjang string pada kolom CatName SM: Error membaca data string pada kolom CustName [INFO] Stats Customer: rows=136, blocks=3 [INFO] Stats Purchase: rows=164, blocks=2 === Storage Manager system test passed === BufferManager: Flushing all dirty pages before shutdown... </pre>

e. Pengujian untuk komponen *failure recovery*

Description	System test untuk memvalidasi kemampuan Failure Recovery Manager dalam menangani crash recovery dengan kombinasi committed dan uncommitted transactions. Test ini mensimulasikan skenario dimana beberapa transaksi berhasil commit sebelum crash, dan beberapa transaksi masih uncommitted saat crash terjadi. Recovery system harus dapat melakukan REDO untuk
-------------	--

	committed transactions dan UNDO untuk uncommitted transactions.
Goal	Memastikan bahwa Failure Recovery Manager dapat: 1. Melakukan REDO (replay) untuk transaksi yang sudah COMMIT sebelum crash 2. Melakukan UNDO (rollback) untuk transaksi yang belum COMMIT saat crash 3. Mempertahankan ACID properties setelah recovery 4. Mengembalikan database ke consistent state setelah unexpected system failure
Method	-- Step 1: Create table -- Step 2: Transaction T1 (akan di-COMMIT) -- Step 3: Verify T1 -- Step 4: Transaction T2 (TIDAK akan di-COMMIT) -- Step 5: Verify before crash -- Step 6: Kill terminal -- Step 7: Restart /mdbms_client -- Step 8: Verify
Success Criterion	Data dari T1 (committed) harus ADA di database setelah recovery Data dari T2 (uncommitted) harus TIDAK ADA di database setelah recovery UPDATE operation dari T2 harus DI-UNDO (nilai kembali ke sebelum update) Tidak ada data corruption atau inconsistency Database dalam state yang valid dan dapat menerima query baru Console menampilkan: "FASE REDO" dengan 2 operations (INSERT dari T1) "FASE UNDO" dengan 2 operations (UPDATE dan INSERT dari T2)
Input	-- Step 1: Create table BEGIN TRANSACTION; CREATE TABLE Student (StudentID INTEGER, FullName VARCHAR 100, GPA FLOAT, PRIMARY KEY StudentID); COMMIT -- Step 2: Transaction T1 (akan di-COMMIT)

	<pre> BEGIN TRANSACTION; INSERT INTO Student VALUES (1, 'Alice Johnson', 3.85); INSERT INTO Student VALUES (2, 'Bob Smith', 3.42); COMMIT; -- Step 3: Verify T1 SELECT * FROM Student; -- Step 4: Transaction T2 (TIDAK akan di-COMMIT) BEGIN TRANSACTION; INSERT INTO Student VALUES (3, 'Charlie Brown', 3.21); UPDATE Student SET GPA = 4.00 WHERE StudentID = 1; -- Step 5: Verify before crash SELECT * FROM Student; # Terminal 2 ps aux grep mdbms kill -9 <PID> # Restart ./build/src/mdbms_client # Verify SELECT * FROM Student; </pre>
Expected Output	Transaction yang belum dicommity harus diredo dan undo. Hasilnya harusnya: mdbms> SELECT * FROM Student; <pre> +-----+-----+-----+ StudentID FullName GPA +-----+-----+-----+ 1 Alice Johnson 3.85 2 Bob Smith 3.42 +-----+-----+-----+ Total rows: 2 </pre>
Results	SUCCESS

	<pre>+-----+ StudentID FullName GPA +-----+ 1 Alice Johnson 3.85 2 Bob Smith 3.42 +-----+</pre>
--	---

Part IV. Distribusi Workload

Tuliskan nama-nama anggota yang berkontribusi. Apabila ada anggota yang tidak berkontribusi, tidak perlu dicantumkan.

1. Query Processor

NIM	Nama	Workload
13523067	Benedict Presley	Pembuatan laporan, inisialisasi komponen, integrasi supergroup, header file + struct (types.h), implementasi terminal UI, implementasi method <code>commit_transaction()</code> , <code>abort_transaction()</code> , <code>execute_drop_table()</code> , <code>execute_create_table()</code> , unit testing, bug fixing
13523109	Haegen Quinston	Pembuatan laporan, implementasi method <code>execute_query()</code> , <code>execute_insert()</code> , <code>execute_delete()</code> , <code>apply_table_aliases()</code> , <code>resolve_aliased_column()</code> , <code>get_table_from_alias()</code> , unit testing, bug fixing
13523091	Carlo Angkisan	Pembuatan laporan, implementasi method <code>execute_select()</code> , <code>begin_transaction()</code> , <code>apply_where_clause()</code> , <code>apply_order_by()</code> , <code>apply_limit()</code> , <code>execute_join()</code> , <code>execute_update()</code> , unit testing, bug fixing
13523037	Buege Mahara Putra	-

2. Query Optimizer

NIM	Nama	Workload
13523087	Grace Evelyn Simon	Pembuatan laporan, implementasi <i>query cost</i>
13523053	Sakti Bimasena	Pembuatan laporan, implementasi <i>query tree</i>
13523025	Joel Hotlan Haris Siahaan	Pembuatan laporan, implementasi <i>query optimizer</i>
13523121	Ahmad Wicaksono	Pembuatan laporan, implementasi <i>query parser</i>

3. Concurrency Control Manager

NIM	Nama	Workload
13522154	Chelvadinda	Pembuatan laporan, inisiasi struktur data, <i>class diagram</i>
13523001	Wardatul Khoiroh	Pembuatan laporan, inisiasi struktur data, <i>class diagram</i> , implementasi <i>Timestamp based</i>
13523029	Bryan Ho	Pembuatan laporan, inisiasi struktur data, <i>class diagram</i> , implementasi <i>Timestamp based, testing</i>
13523031	Rafen Max Alessandro	Pembuatan laporan, inisiasi struktur data, <i>class diagram</i> , implementasi <i>two phase locking, testing</i>
13523073	Alfian Hanif Fitria Yustanto	Pembuatan laporan, inisiasi struktur data, <i>class diagram</i> , implementasi <i>two phase locking, testing</i>

4. Storage Engine

NIM	Nama	Workload
13523013	Nathaniel Jonathan Rusli	Pembuatan laporan, implementasi, BufferManager, delete_block(), serialize()
13523043	Najwa Kahani Fatima	Pembuatan laporan, implementasi set_index(), hash index implementation
13523085	Muhammad Jibril Ibrahim	Pembuatan laporan, implementasi get_stats(), B+ Tree index Implementation, BufferManager
13523095	Rafif Farras	Pembuatan laporan, implementasi read_block(), write_block()

5. Failure Recovery

NIM	Nama	Workload
13523033	Alvin Christopher Santausa	Pembuatan laporan, integrasi, debugging, memastikan semua berjalan dengan benar.
13523111	Muhammad Iqbal Haidar	Pembuatan laporan, implementasi fitur penulisan/pembacaan TableSchema dalam bentuk binary ke/dari file.
13523117	Ferdin Arsenarendra Purtadi	Pembuatan laporan, implementasi fungsi write_log, menerjemahkan ExecutionResult dari Query Processor menjadi logEntry.
13523119	Reza Ahmad Syarif	Pembuatan laporan, implementasi fitur create and drop table recovery

